

Verificación funcional de un FIFO asíncrono mediante UVM.

Comprobación de propiedades de cruce de dominios de reloj

Enmanuel Sancho Quintanilla ¹

Abstract—Este trabajo documenta la construcción de un entorno de verificación funcional en UVM para `prim_fifo_async`, el FIFO asíncrono de dos dominios de reloj del proyecto OpenTitan. El punto de partida no es el RTL sino el contrato de interfaz: de él se derivan trece propiedades y diez funcionalidades que constituyen la especificación verificable frente a la que se contrasta el diseño. Sobre esa base se construye un banco con dos agentes independientes, uno por dominio, comprobación automática mediante scoreboard y aserciones SVA alojadas en las interfaces, y un plan de cobertura de catorce puntos cuyo criterio de cierre se fija antes de escribir el banco y no se relaja después.

La regresión final alcanza los sesenta y seis bins del plan en cincuenta y cuatro corridas sin fallos. El diseño no reveló defectos; el entorno sí reveló siete, entre ellos doce aserciones desactivadas en silencio y un comprobador que acusaba al diseño de un incumplimiento inexistente. Ese contraste articula la tesis del documento: un comprobador que nunca ha fallado no está verificado, y distinguir el silencio correcto del silencio averiado exige presentarle deliberadamente la violación que dice detectar. Se documentan asimismo las decisiones de alcance, las técnicas identificadas y no aplicadas, y las limitaciones del entorno de simulación empleado.

I. INTRODUCCIÓN

A. Contexto y motivación

La verificación funcional es fundamental en el desarrollo de un circuito integrado. A diferencia del diseño, donde el objetivo es que el circuito haga lo que debe, en verificación el objetivo es **demostrar que no hace nada que no deba**, lo que exige recorrer un espacio de comportamiento mucho mayor que el conjunto de casos que el diseñador tuvo en mente.

Este proyecto aplica la metodología UVM a un caso donde esa distancia es especialmente visible: un FIFO asíncrono. Un FIFO síncrono es un componente sencillo y bien acotado; introducir un segundo dominio de reloj cambia la naturaleza del problema. Aparecen comportamientos que no existen en un diseño síncrono (latencia variable entre dominios, observabilidad parcial del estado desde cada lado) y con ellos una clase de propiedades que solo tienen sentido cuando dos relojes no guardan relación entre sí.

Este trabajo nace de la necesidad de adquirir experiencia práctica en verificación funcional con UVM. Se trata, por tanto, de un proyecto con propósito formativo, y esa condición determina deliberadamente cómo está redactado el documento: se explican decisiones y fundamentos que en un entorno profesional se darían por supuestos. La elección es intencionada.

Documentar por qué se toma cada decisión, y no únicamente cuál se tomó, es lo que permite someter un trabajo de verificación a revisión.

B. Objetivos

Objetivo general: Construir un entorno de verificación funcional completo en UVM para un FIFO asíncrono, y alcanzar un criterio de cierre de cobertura definido de antemano.

Objetivos específicos:

- 1) Derivar una especificación verificable del DUT a partir de su contrato de interfaz y de la literatura de referencia sobre cruce de dominios de reloj [1].
- 2) Construir un testbench UVM con dos agentes independientes, uno por dominio de reloj.
- 3) Implementar comprobación automática mediante scoreboard con modelo de referencia y aserciones SVA.
- 4) Definir y alcanzar un plan de cobertura funcional con criterio de cierre explícito.
- 5) Documentar el proceso, las decisiones tomadas y las limitaciones del entorno.

C. Alcance

Dentro del alcance:

- Verificación funcional por simulación RTL.
- Comprobación del protocolo de las interfaces mediante SVA.
- Comprobación de integridad de datos: orden, ausencia de pérdidas y de duplicados.
- Comprobación del reporte de ocupación en ambos dominios.
- Cobertura funcional con criterio de cierre definido.

Fuera del alcance:

- Resolución de metaestabilidad en los sincronizadores. No es posible verificar mediante simulación RTL y se justifica en el apartado III-G.
- Verificación estructural de CDC mediante herramientas de análisis estático.
- Análisis temporal, síntesis e implementación física.
- Verificación formal.

D. Estructura del documento

El documento sigue el orden en que se construyó el trabajo, que es también el orden en que conviene leerlo: primero qué debe verificarse, después con qué infraestructura, y por último qué se obtuvo.

El apartado II describe el diseño bajo prueba: su interfaz, su principio de funcionamiento y la referencia clásica en la que se apoya. El apartado III constituye la especificación del trabajo, y reúne las funcionalidades a verificar, las

propiedades que deben cumplirse siempre, el plan de cobertura, el plan de pruebas y el criterio de cierre. El apartado IV documenta la construcción del entorno de verificación, y el V la estrategia de comprobación, es decir el reparto entre aserciones y modelo de referencia.

Los resultados se recogen en el apartado VI: defectos encontrados, regresión multi-semilla, cobertura final y matriz de trazabilidad. El apartado VII reúne las restricciones que impuso el simulador y las decisiones de arquitectura que se derivaron de ellas, y el VIII contrasta los objetivos con lo obtenido y expone las líneas de trabajo futuro.

Los apartados III y VI son los dos extremos de un mismo hilo. El primero declara qué debe verificarse y bajo qué criterio se dará el trabajo por cerrado; el segundo acredita, mediante la matriz de trazabilidad, que cada declaración tiene un mecanismo de comprobación, una evidencia localizable en el código y una prueba que la ejercita.

No existe un apartado dedicado exclusivamente a la cobertura. El plan de cobertura forma parte de la especificación y se expone en el apartado III; su implementación se documenta junto al resto de los componentes del entorno en el IV, y sus resultados en el VI.

E. Guía de lectura

El documento está escrito para consultarse además de para leerse. Quien disponga de poco tiempo puede recorrer tres itinerarios, según lo que quiera comprobar.

Qué se propuso y si lo consiguió, en pocos minutos: el criterio de cierre del apartado III-F, fijado antes de construir el banco; el resultado de cobertura del apartado VI-G; y el contraste entre ambos en el apartado VIII-A.

Si el banco es de fiar, que es la pregunta pertinente cuando el diseño no ha revelado defectos: la validación de los comprobadores por inyección de fallos del apartado VI-C, el mecanismo de veredicto del apartado VII-C, y la matriz de trazabilidad del apartado VI-H, que acredita que ninguna propiedad declarada quedó sin comprobar.

Cómo se trabajó: los siete defectos encontrados en el propio entorno de verificación, en el apartado VI-B, con la medición que llevó a cada uno; y la recomendación de trabajo futuro del apartado VIII-C.1, que nace de dos de ellos.

Los apartados IV y V documentan la construcción con detalle de implementación. Están escritos para quien pregunte *cómo se hizo esto* y no para leerse de corrido, de modo que los títulos de sus subapartados se han redactado para poder localizarse en el índice.

F. Disponibilidad del código fuente

El entorno de verificación que este documento describe es público, y se recomienda tenerlo a la vista durante la lectura de los apartados IV y V, que remiten a clases y ficheros concretos.

Repositorio:

github.com/sancholl/async-fifo-uvm-env

Contiene las interfaces con sus aserciones, los dos agentes, el entorno con scoreboard y cobertura, el catálogo de pruebas, el guion de regresión y el consolidador de cobertura,

bajo licencia Apache-2.0. Incluye además el RTL del diseño bajo prueba, cuya procedencia se detalla en el apartado II-B.

II. EL DUT, *Design Under Test*

A. Descripción funcional

Un FIFO (*First In, First Out*) es una cola: los datos salen en el mismo orden en que entraron. Su función es desacoplar a un productor de un consumidor que trabajan a ritmos distintos, absorbiendo las diferencias instantáneas de velocidad sin perder información.

Un FIFO **asíncrono** añade una condición: el proveedor y el consumidor están gobernados por relojes **independientes**, sin ninguna relación de frecuencia ni de fase entre ellos. Esto lo convierte en el mecanismo estándar para transferir datos entre dos dominios de reloj de un mismo chip.

El DUT verificado es `prim_fifo_async`, la implementación de FIFO asíncrono de la librería de primitivas del proyecto OpenTitan [5].

B. Procedencia y licencia

El diseño no es original de este trabajo. Se ha tomado sin modificaciones de:

- **Proyecto:** OpenTitan (lowRISC) [5]
- **Fichero:** `hw/ip/prim/rtl/prim_fifo_async.sv`
- **Licencia:** Apache 2.0

El RTL se redistribuye en el repositorio de este trabajo, bajo el directorio `opentitan/` y amparado por esa misma licencia, con sus cabeceras de copyright intactas y **sin modificación alguna**. Procede de una revisión **fijada** y no de la rama principal: los resultados que recoge este documento se obtuvieron contra esa revisión exacta, y seguir a la rama principal haría que el banco verificase un diseño distinto del aquí descrito. La procedencia fichero a fichero, y el procedimiento para comprobar que las copias son literales, constan en el propio repositorio.

Se eligió un diseño de producción, empleado en silicio real, en lugar de escribir uno propio. Ya que el objeto de este trabajo es el **entorno de verificación**, y verificar un diseño que uno mismo ha escrito introduce un sesgo evidente: se tiende a comprobar lo que se sabe que se implementó, no lo que la especificación exige.

C. Interfaz: puertos y parámetros

Parámetros, en la tabla I:

La restricción de potencia de 2 está impuesta por una aserción de elaboración en el propio RTL, y es necesaria para que la aritmética de punteros en código Gray funcione.

1) **Puertos del dominio de escritura:** Se recogen en la tabla II.

2) **Puertos del dominio de lectura:** Se recogen en la tabla III.

Ambas interfaces siguen un protocolo de handshake `valid/ready`. Donde la transferencia ocurre en el flanco de reloj positivo en el que ambas señales están activas simultáneamente.

Parámetro	Descripción
Width	Ancho del dato en bits
Depth	Número de posiciones (Espacios de memoria de la cola). Debe ser potencia de 2
OutputZeroIfEmpty	Si se habilita fuerza <code>rdata_o</code> a cero cuando el FIFO está vacío
OutputZeroIfInvalid	Si se habilita fuerza <code>rdata_o</code> a cero cuando <code>rvalid_o</code> está bajo
DepthW	Derivado: $\lceil \log_2(\text{Depth}+1) \rceil$, ancho de los registros que muestran la ocupación de la cola

TABLE I
PARÁMETROS DE CONFIGURACIÓN DE `PRIM_FIFO_ASYNC`.

Puerto	Dir.	Descripción
<code>clk_wr_i</code>	in	Reloj del dominio de escritura
<code>rst_wr_ni</code>	in	Reset asíncrono, activo a nivel bajo
<code>wvalid_i</code>	in	El productor ofrece un dato
<code>wready_o</code>	out	El FIFO puede aceptarlo
<code>wdata_i</code>	in	Dato a escribir
<code>wdepth_o</code>	out	Ocupación estimada desde el dominio de escritura

TABLE II
PUERTOS DEL DOMINIO DE ESCRITURA.

Puerto	Dir.	Descripción
<code>clk_rd_i</code>	in	Reloj del dominio de lectura
<code>rst_rd_ni</code>	in	Reset asíncrono, activo a nivel bajo
<code>rvalid_o</code>	out	Hay un dato disponible
<code>rready_i</code>	in	El consumidor lo acepta
<code>rdata_o</code>	out	Dato leído
<code>rdepth_o</code>	out	Ocupación estimada desde el dominio de lectura

TABLE III
PUERTOS DEL DOMINIO DE LECTURA.

Obsérvese que **el diseño expone un reset por dominio**, ambos asíncronos y activos a nivel bajo. Esa duplicidad es una característica estructural de la interfaz, y no implica que los dos resets puedan gobernarse por separado: el estado del FIFO es la diferencia entre dos punteros que viven en dominios distintos, de modo que reiniciar uno solo deja al diseño en un estado que su especificación no cubre. El contrato que este trabajo asume al respecto se establece en PROP-12, en el apartado III-C.

D. Principio de funcionamiento

1) *Memoria y punteros:* El FIFO se construye sobre un array de `Depth` posiciones de memoria y dos índices:

- `wptr`: Posición donde se escribirá el próximo dato.
- `rprr`: Posición de donde se leerá el próximo dato.

Cada escritura adelanta `wptr`; cada lectura adelanta `rprr`. El puntero de lectura persigue permanentemente al de escritura. En el RTL, el array es `storage[Depth]` y los punteros direccionan sobre él tomando únicamente sus bits bajos, en las posiciones $[0+:\lceil \log_2(\text{Depth}) \rceil - 1]$

2) *Distinción entre lleno y vacío: el bit de vuelta:* Los índices, por sí solos, no bastan para distinguir las dos

condiciones límite. Al arrancar, ambos punteros valen cero y el FIFO está vacío. Tras escribir `Depth` datos sin leer ninguno, el puntero de escritura ha dado una vuelta completa y **vuelve a coincidir** con el de lectura, pero ahora el FIFO está lleno. La comparación `wptr == rprr` es cierta en ambos casos.

La solución, establecida en la referencia clásica sobre esta arquitectura [1], es dar a los punteros **un bit más** del necesario para direccionar la memoria:

```
// cantidad de bits que direccionan
// storage
PTRV_W = $clog2(Depth)
// cantidad de bits del puntero completo,
// con el bit de vuelta
PTR_WIDTH = PTRV_W + 1
```

El bit adicional no participa en el direccionamiento (la memoria se indexa descartándolo) sino que cuenta la **paridad de vueltas** dadas sobre el array. Con él:

- **Vacío:** Los punteros coinciden por completo, incluido el bit de vuelta.
- **Lleno:** Coinciden los bits de dirección, pero difiere el bit de vuelta.

En el RTL esto se implementa comparando los punteros directamente para la condición de vacío, y comparándolos tras invertir el bit más significativo de uno de ellos (mediante una máscara XOR) para la condición de lleno.

3) *Cruce de dominios: Gray Code:* Cada dominio necesita conocer el puntero del otro para decidir si puede escribir o leer. Ese puntero debe cruzar entre relojes independientes, y ahí aparece el problema.

Cuando un valor de varios bits cruza un dominio, cada bit se captura por separado. Si varios bits cambian simultáneamente, unos pueden capturarse con el valor nuevo y otros con el viejo, produciendo un valor que **el contador nunca tuvo**. Un contador binario que pasa de 0111 a 1000 cambia cuatro bits a la vez; una captura desafortunada puede dar 1111 o 0000.

La codificación Gray resuelve esto [1]: entre dos valores consecutivos **cambia exactamente un bit**. Al haber un único bit en transición, cualquier captura devuelve necesariamente el valor anterior o el siguiente, nunca un tercero espurio. El error queda acotado a un desfase temporal, que es tolerable, en lugar de a una corrupción de valor, que no lo sería.

La garantía es por transición, no por intervalo de muestreo. Conviene precisar el alcance de esa propiedad, porque de ella depende que el diseño admita relaciones de frecuencia arbitrarias, que es una de las garantías que la arquitectura de referencia [1] persigue. El código Gray no asegura que entre dos muestras consecutivas del dominio de destino cambie un solo bit; asegura que entre dos valores consecutivos *del contador* cambie un solo bit.

La distinción importa cuando un dominio es sensiblemente más rápido que el otro. Si el reloj de escritura duplica al de lectura, entre dos flancos de lectura el puntero puede avanzar dos posiciones, y el destino observará un valor que difiere en dos bits del anterior que muestreó. Eso no compromete la

captura: las dos transiciones ocurrieron en instantes distintos, separados por un periodo completo del reloj de origen, de modo que en el momento de la captura hay a lo sumo **un** bit inestable. El flop lo resuelve hacia el valor viejo o hacia el nuevo, y en ambos casos el resultado es un valor que el contador efectivamente tuvo.

El efecto real de una relación de frecuencias desfavorable es que el destino **se salta valores intermedios**: observa el puntero en una posición y después en otra dos más adelante, sin pasar por la de en medio. Para un puntero de FIFO eso es inocuo, porque se trata de un **nivel monótono y no de un evento**: al receptor no le hace falta enterarse de cada paso, sino únicamente de hasta dónde ha llegado el otro dominio. Saltarse valores equivale a observar el puntero con más retraso, que es la latencia variable que el diseño tolera por construcción y que este trabajo formaliza en PROP-09.

El contraste aclara el alcance de la garantía: **un pulso** que cruzase de dominio sí impondría una restricción sobre las frecuencias, porque un pulso más corto que el periodo de destino puede perderse por completo y exige handshake o estiramiento. Un puntero codificado en Gray no la impone, precisamente por ser un nivel.

El RTL mantiene ambas representaciones de cada puntero, binaria y Gray, e incluye funciones explícitas de conversión (`dec2gray` y `gray2dec`). Solo la versión Gray cruza el dominio; en destino se reconvierte a binario antes de compararla.

El reflejo entre vueltas. Aplicar la conversión Gray directamente sobre el puntero completo no basta, y el punto donde falla es precisamente la vuelta. Por eso `dec2gray` aplica una transformación distinta según el valor del bit de vuelta: cuando vale cero, los bits de dirección se convierten a Gray de la forma habitual; cuando vale uno, la función los sustituye antes por su complemento respecto a `Depth`, de modo que la cuenta no se reinicia en la segunda vuelta sino que se **refleja**, recorriendo las mismas posiciones en orden inverso. El bit de vuelta se propaga sin modificar y ocupa el bit más significativo del resultado.

El efecto se aprecia mejor sobre la secuencia completa. Para `Depth = 4`, con punteros de tres bits, la tabla IV:

dec	0	1	2	3	4	5	6	7
gray	000	001	011	010	110	111	101	100

TABLE IV

CÓDIGO GRAY DE TRES BITS: ENTRE VALORES CONSECUTIVOS CAMBIA UN ÚNICO BIT.

Entre cualesquiera dos valores contiguos cambia un solo bit, **incluida la vuelta**. En el paso de 3 a 4 solo cambia el bit más significativo, y lo mismo ocurre al cerrarse el ciclo de 7 a 0: la secuencia es un código Gray cíclico completo.

Esa es exactamente la función del reflejo. Sin él, el valor 3 se codificaría como 010 y el 4 como 100, con dos bits cambiando a la vez justo en el instante de la vuelta, que es el escenario que la codificación Gray existe para evitar. Al reflejar, los bits de dirección del último valor de una vuelta

y los del primero de la siguiente **coinciden**, y el único bit que queda cambiando es el de vuelta.

La consecuencia para el cruce de dominios es que la vuelta no constituye un caso especial: una captura que coincida con ella devuelve el último valor de la vuelta anterior o el primero de la nueva, igual que en cualquier otra transición.

4) *Sincronización: la cadena de dos flip-flops*: El código Gray resuelve la coherencia entre bits, pero no el problema de cada bit individual. Un flip-flop requiere que su entrada permanezca estable durante una ventana alrededor del flanco de reloj. Cuando la señal procede de un dominio sin relación con el reloj de captura, esa condición no puede garantizarse: los flancos se cruzan en cualquier posición y antes o después la transición cae dentro de la ventana prohibida. El flop entra entonces en **metaestabilidad**, con su salida en un nivel indefinido durante un tiempo impredecible, tras el cual se resuelve hacia un valor u otro de forma no determinista.

El riesgo no es el retardo, sino la incoherencia: si esa salida indefinida alimentara directamente la lógica, distintas puertas podrían interpretarla de forma distinta a partir del mismo cable.

La mitigación estándar [1] es una cadena de dos flip-flops en el dominio de destino:

- El **primero** absorbe la incertidumbre. Puede quedar metaestable, y su salida no alimenta nada salvo el segundo flop.
- El **ciclo completo** que transcurre entre ambos es el tiempo concedido para que esa metaestabilidad se resuelva.
- El **segundo** captura un valor ya estable, sea el nuevo o el anterior, pero siempre un nivel lógico limpio.

En el DUT esta función la cumple el módulo `prim_flop_2sync`, instanciado una vez por sentido de cruce.

Debe subrayarse que esta técnica **no elimina el riesgo, lo hace improbable**. Existe siempre una probabilidad no nula de que la metaestabilidad persista más de un ciclo. Lo que se hace en la práctica es calcular el tiempo medio entre fallos, que crece de forma exponencial con el tiempo de asentamiento concedido, y dimensionar el número de etapas para que ese tiempo resulte irrelevante frente a otras fuentes de fallo del sistema. **La imposibilidad de capturar una señal asíncrona con probabilidad nula de metaestabilidad es un resultado conocido, no una carencia de esta implementación.**

5) *Ocupación: `wdepth_o` y `rdepth_o`*: El DUT expone la ocupación del FIFO en **ambos** dominios. Los dos valores pueden diferir en un instante dado, y esa diferencia es una propiedad estructural del diseño, no un defecto.

La causa es la latencia de sincronización, y actúa siempre, con independencia de que se produzca o no metaestabilidad. Cada dominio calcula la ocupación empleando **su propio puntero, actualizado, y el del otro dominio, retrasado** varios ciclos por la cadena de sincronización, como resume la tabla V:

Ambas estimaciones son erróneas, y ambas lo son **hacia el lado seguro**:

	Puntero propio	Puntero remoto	Cree que...	Resultado
wdepth_o	escritura, actual	lectura, retrasado	se ha leído menos de lo real	sobreestima la ocupación
rdepth_o	lectura, actual	escritura, retrasado	se ha escrito menos de lo real	subestima la ocupación

TABLE V

CARÁCTER CONSERVADOR DE LOS DOS REPORTES DE OCUPACIÓN:
CADA DOMINIO YERRA EN LA DIRECCIÓN SEGURA.

- El escritor cree que hay menos hueco libre del que hay, y deja de escribir antes de lo estrictamente necesario. **Nunca desborda.**
- El lector cree que hay menos datos de los que hay, y no intenta leer de más. **Nunca lee una posición sin escribir.**

El coste es de rendimiento: puntualmente el FIFO podría aceptar un dato más o entregar uno antes. A cambio, la corrupción de datos resulta estructuralmente imposible.

De esta asimetría se deriva una de las propiedades centrales de este trabajo, formalizada como **PROP-07** en el apartado III-C.

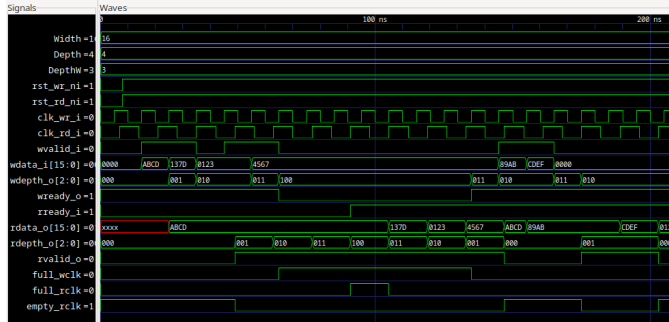


Fig. 1. Diagrama de señales del funcionamiento del FIFO Asíncrono de Open Titan.

E. Relación con la referencia clásica

El trabajo de referencia sobre esta clase de diseño es *Simulation and Synthesis Techniques for Asynchronous FIFO Design*, de Clifford Cummings [1], que establece la arquitectura de punteros en Gray con sincronizadores de dos etapas empleada aquí.

La implementación de OpenTitan se aparta del artículo en un punto: Cummings compara los punteros **directamente en su representación Gray**, empleando una comparación específica sobre los bits más significativos, mientras que `prim_fifo_async` emplea Gray únicamente para el cruce y **reconvierte a binario** en el dominio de destino antes de comparar.

Ambas soluciones son correctas. La de OpenTitan produce una lógica de comparación más legible (la condición de vacío es una igualdad directa) a costa de la lógica de conversión.

III. PLAN DE VERIFICACIÓN

A. Metodología: ¿Cómo se derivó la especificación?

`prim_fifo_async` es una primitiva de librería y **no dispone de una especificación publicada**. La documentación de OpenTitan describe la arquitectura del sistema de primitivas, no el comportamiento individual de cada módulo.

Esto plantea un problema metodológico que conviene clarificar, porque determina la validez de todo el trabajo. De forma inmediata derivar el comportamiento esperado leyendo el RTL podría parecer el camino a seguir. Sin embargo hacerlo invalidaría la verificación: si el testbench se construye a partir de la implementación, cualquier defecto del diseño queda codificado como comportamiento esperado. El entorno pasaría siempre y no aportaría información.

La especificación se ha derivado, por tanto, de tres fuentes independientes del RTL:

- 1) **El contrato de la interfaz.**: Los puertos y el protocolo `valid/ready` definen obligaciones observables desde fuera, independientes de cómo esté implementado el interior.
- 2) **La definición funcional de un FIFO.**: Orden preservado, ausencia de pérdidas y de duplicados, imposibilidad de escribir cuando está lleno o leer cuando está vacío. Son propiedades del concepto, no de esta implementación.
- 3) **La literatura de referencia sobre CDC**: En particular el artículo de Cummings [1], de donde proceden las propiedades asociadas al cruce de dominios.

Por tanto el RTL se ha empleado exclusivamente para dos fines: **conocer la interfaz** y **saber qué señales internas observar** al depurar. En ningún caso como fuente del comportamiento esperado.

B. Funcionalidades a verificar

La tabla VI enumera las diez funcionalidades derivadas del contrato, con las propiedades y los puntos de cobertura que acreditan cada una.

Id	Funcionalidad	Propiedades	Cobertura
F-01	Escritura de datos con handshake <code>wvalid/wready</code>	PROP-04, 05, 10	CP-01, CP-04
F-02	Lectura de datos con handshake <code>rvalid/rready</code>	PROP-04, 05, 11	CP-02
F-03	Preservación del orden FIFO	PROP-01, 02, 03	— (nota 1)
F-04	Señalización y comportamiento en condición de lleno	PROP-10	CP-06, CP-14
F-05	Señalización y comportamiento en condición de vacío	PROP-11	CP-07, CP-15
F-06	Reporte de ocupación en el dominio de escritura	PROP-07, 08	CP-04, CP-12
F-07	Reporte de ocupación en el dominio de lectura	PROP-07, 08	CP-05, CP-07, CP-13
F-08	Operación con relojes de frecuencias arbitrarias	PROP-09	CP-10, CP-12, CP-13, CP-14, CP-15
F-09	Reset simultáneo de ambos dominios	PROP-06, 12, 13	CP-03, CP-09
F-10	Vuelta de los punteros	— (nota 2)	CP-08

TABLE VI

FUNCIONALIDADES A VERIFICAR, CON LAS PROPIEDADES Y LOS PUNTOS DE COBERTURA QUE LAS ACREDITAN.

Nota 1: F-03 no tiene punto de cobertura propio, y es correcto. La preservación del orden no es un escenario que se alcance o no: es una propiedad que se comprueba en *cada* transacción de lectura. Todo el tráfico la ejercita, de modo que un punto de cobertura dedicado no distinguiría situaciones. Lo que sí interesa medir son las condiciones bajo las cuales se ejercita, y eso es precisamente CP-04 y los cruces.

Nota 2: F-10 no tiene propiedad propia, y también es correcto. La corrección de la vuelta de punteros no se observa directamente: se manifiesta como pérdida de orden o de integridad, que PROP-01 a PROP-03 ya detectan. Si el bit de vuelta o la codificación Gray fallasen, el scoreboard reportaría discrepancia de dato. F-10 necesita por tanto cobertura (para garantizar que el escenario se alcanza) pero no una propiedad adicional.

Esta tabla es la matriz de trazabilidad del proyecto. Su utilidad no es documental sino diagnóstica: **una funcionalidad sin propiedad no se comprueba, y una funcionalidad sin cobertura no se sabe si se ha ejercitado.**

C. Propiedades verificables

1) Integridad de datos:

- **PROP-01: Preservación del orden.** Los datos se entregan por el puerto de lectura en el mismo orden en que fueron aceptados por el puerto de escritura.

Comprobación: scoreboard con modelo de referencia.

Cobertura: CP-04.

- **PROP-02: Ausencia de pérdidas.** Todo dato aceptado en escritura es entregado exactamente una vez en lectura, salvo reset intermedio.

Comprobación: scoreboard, comparación de cardinalidad al final del test.

- **PROP-03: Ausencia de duplicados.** Ningún dato se entrega más de una vez.

Comprobación: scoreboard.

2) Protocolo de interfaz:

- **PROP-04: Estabilidad de `valid`.** Una vez `wvalid_i` está activo, permanece activo hasta que se produce la transferencia. De forma análoga en el dominio de lectura se comprueba para `rvalid_o`.

Comprobación: SVA en la interfaz de escritura y en la interfaz de lectura.

- **PROP-05: Estabilidad del dato.** Mientras `wvalid_i` está activo y `wready_o` inactivo, `wdata_i` permanece constante.

Comprobación: SVA en la interfaz de escritura.

- **PROP-06: Inactividad durante reset.** No se produce ninguna transferencia mientras el reset del dominio correspondiente está activo. Dado que la transferencia se define por el contrato `valid/ready`, la propiedad se descompone en obligaciones separadas por dominio y por sujeto:

- (Escritura, agente) El agente no inicia ni mantiene una transferencia durante el reset: `wvalid_i` permanece inactiva.
- (Escritura, DUT) El DUT no altera su estado observable durante el reset: `wdepth_o` permanece en cero.
- (Lectura, DUT) El DUT no inicia ni mantiene una transferencia durante el reset: `rvalid_o` permanece inactiva.
- (Lectura, DUT) El DUT no altera su estado observable durante el reset: `rdepth_o` permanece en cero.

Comprobación: SVA en ambas interfaces.

Notas de implementación:

- Ni `wready_o` ni `rvalid_o` están gateadas por reset; ambas son combinacionales sobre una comparación de punteros. El reset deja ambos punteros iguales, y sobre punteros iguales los predicados dan resultados opuestos: *vacío* es cierto (luego `rvalid_o` baja) y *lleno* es falso (luego `wready_o` sube). La asimetría procede de los predicados, no del tratamiento del reset.

- En consecuencia, PROP-06 no debe interpretarse como una obligación sobre `wready_o`. Y la subpropiedad (a) es necesaria precisamente porque `wready_o` está alta durante el reset: un agente que condujera `wvalid_i` vería un handshake aparentemente completado mientras el puntero, en reset, no

avanza. El dato se perdería sin ninguna señal de error.

- No se exige que `rready_i` esté inactiva durante el reset. A diferencia de `wvalid_i`, que es una petición, `rready_i` es un permiso, y mantenerlo activo de forma permanente es un comportamiento legítimo del contrato. Exigirlo sobre-restringiría la especificación sin ganancia: con `rvalid_o` baja no puede producirse transferencia alguna.

3) Cruce de dominios:

- **PROP-07: Comportamiento conservador de los reportes de ocupación.** En todo evento se cumple:

$$rdepth_o \leq ocupacion_real \leq wdepth_o \quad (1)$$

Donde entendemos por evento:

- Cuando llega un ítem del monitor de escritura: comprueba $wdepth_o \geq ocupacion_real$.
- Cuando llega uno de lectura: comprueba $rdepth_o \leq ocupacion_real$.

Justificación: cada dominio observa el puntero remoto con retardo de sincronización, lo que hace que el escritor sobreestime la ocupación y el lector la subestime. Ambos errores apuntan al lado seguro. Un incumplimiento en cualquiera de los dos sentidos indicaría un fallo real de diseño: sobreestimar el hueco libre permitiría desbordamiento, y sobreestimar los datos disponibles permitiría leer posiciones no escritas.

Comprobación: aserción en el scoreboard, evaluada en cada transacción.

Cobertura: CP-04, CP-05, CP-10, CP-12, CP-13.

- **PROP-08: Cota superior de los reportes.** Ni `wdepth_o` ni `rdepth_o` exceden `Depth`.

Comprobación: SVA.

- **PROP-09: Tolerancia a la latencia variable.** El comportamiento funcional del FIFO es correcto con independencia del número de ciclos que tarde un puntero en cruzar.

Justificación: un sincronizador de dos flops no garantiza un número fijo de ciclos de travesía, de modo que un banco que lo asumiera quedaría ligado a una implementación concreta del cruce. El entorno no debe asumir una latencia fija.

Comprobación: Implícita; el scoreboard no modela latencia de cruce. **La propiedad no se ejerce**, y no por falta de intento: se concluyó que no hay forma de hacerlo con las herramientas de este flujo que acredite nada. El apartado VI-E documenta las medidas que llevan a esa conclusión.

- **PROP-10: Bloqueo por lleno.** `wready_o` está inactivo cuando el FIFO está lleno según la visión del dominio de escritura.

Comprobación: SVA.

Cobertura: CP-06.

- **PROP-11: Bloqueo por vacío.** `rvalid_o` está inactivo cuando el FIFO está vacío según la visión del dominio de lectura.

Comprobación: SVA.

Cobertura: CP-07.

- **PROP-12: Reset** Tras la activación del reset el FIFO queda vacío: no conserva datos escritos con anterioridad, y ambos dominios reportan ocupación nula.

Contrato soportado: El diseño expone un puerto de reset por dominio de reloj (`rst_wr_ni`, `rst_rd_ni`) porque cada biestable debe recibir su reset liberado de forma síncrona respecto a su propio reloj [2], y el módulo no puede sincronizar esa liberación internamente. Los dos puertos corresponden por tanto a **dos entregas simultáneas y asíncronas de un mismo reset lógico**, no a dos dominios de reset independientes. Por tanto se verifica la **activación simultánea** de ambos; su **liberación desfasada** sí está soportada y también se verifica.

Justificación del desfase en la liberación: Durante la ventana entre una liberación y la otra ambos punteros valen cero, de modo que la estructura es consistente vista desde cualquiera de los dos lados: el dominio que sale primero observa el FIFO vacío y no lleno, que es el estado real. La relación entre punteros parte de cero y no puede corromperse.

Fuera de contrato: La activación del reset de *un solo dominio* mientras el otro opera con punteros distintos de cero. El estado de un FIFO asíncrono es la diferencia entre dos punteros; reiniciar uno solo lo destruye, y los reportes de ocupación de ambos lados pasan a carecer de significado. No es un defecto del diseño sino una consecuencia estructural, común a cualquier FIFO asíncrono. Un sistema que necesite apagar un dominio de forma independiente debe vaciar o resetear también el otro, lo que constituye un protocolo de nivel superior ajeno a este módulo.

Comprobación: Scoreboard con manejo explícito de reset. Al detectarse la activación, el modelo de referencia se vacía; los datos en vuelo se consideran perdidos legítimamente, conforme a la salvedad recogida en PROP-02.

- **PROP-13: Liberación síncrona del reset.** El reset de cada dominio se libera en un flanco activo del reloj correspondiente, nunca a mitad de ciclo.

La bibliografía sobre reset asíncrono [2] señala que la mayor parte de los fallos asociados a reset no se producen en la afirmación (donde todos los biestables se reinician simultáneamente y el estado resultante es consistente por construcción) sino en la **liberación**, donde una transición asíncrona puede violar los tiempos de recovery y removal y dejar biestables del mismo banco en estados distintos.

Sujeto de la obligación: El diseño recibe `rst*_ni` como entradas y no controla su generación: la obligación recae sobre el entorno de verificación, igual que en PROP-06a. Se comprueba sobre el agente propio, no sobre el DUT.

Comprobación: SVA en ambas interfaces.

D. Plan de Cobertura

1) *Puntos de cobertura:* El plan completo está en la tabla VII. Los apartados que siguen justifican cada punto.

Id	Punto	Tipo	Bins	Muestreo
CP-01	Espaciado entre escrituras	valor	back_to_back, short_gap, long_gap	cada transacción de escritura
CP-02	Espaciado entre lecturas	valor	back_to_back, short_gap, long_gap	cada transacción de lectura
CP-03	Estado del bus en el reset	valor	during_idle, during_transfer	activación de reset, por interfaz
CP-04	Ocupación vista por el dominio de escritura	valor	one, half, almost_full, full	cada transacción de escritura
CP-05	Ocupación vista por el dominio de lectura	valor	empty, one, half, almost_full	cada transacción de lectura
CP-06	Condición de lleno	transición	reached, abandoned	cada transacción de escritura
CP-07	Condición de vacío	transición	reached, abandoned	cada transacción de lectura
CP-08	Vuelta de punteros	valor	wr_wrap, rd_wrap	al completarse cada vuelta
CP-09	Ocupación en el momento del reset	valor	while_empty, while_partial, while_full	activación de reset
CP-10	Relación de frecuencias	valor	wr_faster, similar, rd_faster	una vez por corrida
CP-12	Cruce CP-04 × CP-10	cruce	combinaciones alcanzables	—
CP-13	Cruce CP-05 × CP-10	cruce	combinaciones alcanzables	—
CP-14	Cruce CP-06 × CP-10	cruce	las dos transiciones bajo cada relación	—
CP-15	Cruce CP-07 × CP-10	cruce	las dos transiciones bajo cada relación	—

TABLE VII

PLAN DE COBERTURA: LOS CATORCE PUNTOS, SUS BINS Y SU MOMENTO DE MUESTREO.

Sobre el recuento, que aparece con tres cifras distintas a lo largo del documento. El plan declara **catorce puntos**, CP-01 a CP-15: diez de valor o transición y cuatro cruces. En la instrumentación, CP-03 se recoge por separado en cada agente, de modo que los puntos instrumentados son **quince**, once de valor o transición más los cuatro cruces. Las cifras de los resultados se refieren siempre a los instrumentados.

Los identificadores siguen un orden funcional: **estímulo y protocolo** (CP-01 a CP-03), **estado del diseño** (CP-04 a CP-09), **condiciones del entorno** (CP-10) y **cruces** (CP-12 a CP-15). Los cruces van en el mismo orden que sus operandos: CP-12 y CP-13 cruzan las dos vistas de la ocupación, CP-14 y CP-15 las dos condiciones límite. Ese mismo orden es

el de la tabla de instrumentación del apartado III-D.11, de modo que los puntos aparecen agrupados por el componente que los recolecta.

2) *CP-01 y CP-02:* Espaciado entre transacciones (tanto entre lecturas, como entre escrituras).

Mientras que CP-01 mide la presión que el agente de escritura ejerce sobre el DUT, CP-02 es su equivalente en la interface de lectura, de modo que el plan responde a una pregunta legítima: **¿Se ejercitaron transacciones consecutivas sin hueco?** Los bins son los de la tabla VIII.

Bin	Criterio
back_to_back	ninguna espera entre transferencias consecutivas
short_gap	de 1 a 3 ciclos de espera
long_gap	más de 3 ciclos

TABLE VIII

BINS DE CP-01 Y CP-02: ESPACIADO ENTRE TRANSACCIONES CONSECUTIVAS.

Justificación: El caso back_to_back es el que somete al diseño a la máxima tasa de escritura y el único capaz de alcanzar la condición de lleno por sí solo, sin depender de que el lector se retrase. Es además el patrón bajo el que el puntero de escritura avanza en ciclos consecutivos, que es la condición en la que el cruce de dominios se ejercita con mayor exigencia.

A diferencia de los demás puntos, CP-01 y CP-02 miden **el estímulo, no el diseño**: son cobertura de protocolo, y de ahí su ubicación distinta, expuesta en el apartado III-D.11

3) *CP-03:* ¿En qué estado estaba el bus cuando llegó el reset?

CP-09 mide en qué estado se encontraba **el FIFO** al llegar el reset; CP-03, en qué estado se encontraba **el bus**. Son preguntas distintas y ambas necesarias. Los bins de CP-03 están en la tabla IX.

Bin	Criterio
during_idle	el reset llegó sin transferencia pendiente
during_transfer	llegó con valid activo y la transferencia sin completar

TABLE IX

BINS DE CP-03: ESTADO DEL BUS EN EL INSTANTE DEL RESET.

Justificación: El mismo razonamiento que sostiene CP-09, aplicado al otro lado: un reset que llega con el bus ocioso **no ejerce la ruta de aborto**. El manejador de reset del driver y del monitor termina un proceso que no estaba conduciendo ni recolectando nada, de modo que la parte del manejo de reset con más aristas quedaría sin probar.

Dado que contamos con un mismo punto de cobertura en dos interfaces distintas hacemos la distinción de:

- **CP-03a:** para la cobertura en el agente de escritura.
- **CP-03b:** para referirnos a la cobertura en el agente de lectura.

Sobre la anchura de la ventana: Podría objetarse que una transferencia ocurre en un único flanco y que, por

tanto, el bin *during_transfer* sería casi inalcanzable. La objeción confunde **traspaso** con *transacción*: el traspaso es efectivamente un flanco, pero la transacción del agente abarca desde que afirma *valid* hasta que recibe *ready*, y bajo contrapresión (el FIFO lleno en escritura, vacío en lectura) esa espera dura muchos ciclos. La ventana es ancha, y tanto más cuanto más se ejercitan las condiciones límite, que es precisamente lo que el resto del plan persigue.

Nota de implementación: Es el único punto del plan que **no puede recolectarse de forma puramente transaccional**: requiere conocer el instante del reset y el estado de la señal de *valid* en ese momento, y ninguna de las dos cosas llega por un puerto de análisis.

Se recolecta en el componente de cobertura de cada agente, que implementa la **interface class** *prim_async_fifo_reset_handler* y obtiene la interfaz virtual de su objeto de configuración, igual que el driver y el monitor. El agente ya invoca el manejador sobre todos sus hijos, de modo que el enganche existe y no requiere estructura nueva. El valor se toma del clocking block del monitor, que devuelve el muestreo del último flanco: la pregunta es si había una transferencia pendiente en el ciclo en que llegó el reset, no en el instante exacto.

La excepción se documenta en lugar de ocultarse: el resto de la recolección sigue siendo transaccional, y este punto es el precio de medir un escenario que, de otro modo, solo constaría en el listado de tests.

4) CP-04, CP-05: La ocupación no es un valor único, y por eso son dos puntos. Los dos dominios reportan ocupaciones distintas por la latencia de sincronización (PROP-07): *wdepth_o* es conservador por exceso (puede sobreestimar, nunca subestimar) y *rdepth_o* lo es por defecto. Un plan que hablase de "la ocupación" sin decir cuál mide sería ambiguo, y medir solo una de las dos dejaría sin comprobar la afirmación de PROP-07 de que difieren. CP-04 se mide sobre *wdepth_o* y CP-05 sobre *rdepth_o*, cada uno en la transacción de su propio dominio.

¿Por qué los rangos de bins son distintos?: No es una asimetría arbitraria: es una consecuencia estructural del instante de muestreo. El monitor toma la ocupación *después* de que el traspaso se haya completado, de modo que **ningún dominio puede observar el extremo del que él mismo acaba de alejarse**.

Tras una escritura aceptada hay al menos un elemento dentro, luego *wdepth_o* ≥ 1 y el bin *empty* es inalcanzable en CP-04. Tras una lectura aceptada acaba de liberarse un hueco, luego *rdepth_o* $\leq \text{Depth}-1$ y el bin *full* es inalcanzable en CP-05. En ambos casos la imposibilidad es estructural y no de estímulo: **recuperar el valor extremo exigiría que una transacción del dominio opuesto se sincronizase en un solo ciclo, cosa que los**

sincronizadores de dos biestables impiden por construcción.

Los bins imposibles se omiten en lugar de declararse como huecos justificados. Un bin que no puede llenarse mantiene la cobertura por debajo del 100 % de forma permanente y obliga a repetir la misma explicación en cada informe de regresión; entre los dos puntos, el rango 0 .. Depth queda cubierto por completo.

Sobre la geometría. Los bins asumen $\text{Depth} \geq 4$. Con $\text{Depth} = 2$, *half* coincidiría con *one* y *almost_full*, y el punto habría que redefinirlo. La cabecera *prim_async_fifo_defines.svh* permite sobrescribir la profundidad desde la línea de comandos, de modo que la restricción se hace explícita aquí.

5) Bins de transición de CP-06 y CP-07: ¿Qué significan *reached* y *abandoned*?

CP-06 y CP-07 no cubren valores sino cambios de valor. La diferencia importa y conviene verla sobre una traza:

ciclo:	0	1	2	3	4	5	6	7	8
wdepth_o:	0	1	2	3	4	4	4	3	2
lleno:	0	0	0	0	1	1	1	0	0
						↑		↑	
						reached		abandoned	

Un bin de valor solo registraría "la señal valió 1 en algún momento". Un bin de transición registra el cambio: *reached* es la transición 0 => 1 y *abandoned* la transición 1 => 0.

```
coverpoint fifo_full {
  bins reached = (1'b0 => 1'b1);
  bins abandoned = (1'b1 => 1'b0);
}
```

Justificación: Alcanzar la condición demuestra que el diseño sabe señalarla. **Abandonarla demuestra que se recupera.** Un FIFO que se llena y queda bloqueado para siempre, o que deja la bandera enganchada, cubriría *reached* y pasaría el criterio de cierre sin que nadie hubiera comprobado lo más importante: que la condición límite es transitoria y no terminal.

6) CP-08: ¿Cómo se observa una vuelta de puntero?

Los punteros internos **no son observables** desde las interfaces: solo lo son *wdepth_o* y *rdepth_o*, y la ocupación no revela si un puntero ha dado la vuelta. Observarlos por referencia jerárquica al RTL queda descartado por el mismo criterio que rige las aserciones: la cobertura se define sobre la especificación, no sobre la implementación.

El evento sí es deducible del contrato: un puntero recorre Depth posiciones y avanza una por transacción aceptada, luego **tras Depth escrituras aceptadas el puntero de escritura ha dado necesariamente una vuelta**. El componente de cobertura lleva su propio contador y muestrea el evento cuando cruza un múltiplo de Depth, lo mismo, de forma independiente, para lectura.

Dado que contamos con un mismo punto de cobertura que se debe aplicar a dos dominios de reloj distintos se implementa como dos bins distintos dentro de un mismo coverpoint.

Justificación: La vuelta del puntero es el punto donde la codificación Gray y el bit de vuelta se ejercitan de verdad: es la transición en la que el valor codificado pasa por el cambio que distingue lleno de vacío. Un banco que nunca completa una vuelta deja sin ejercitar el mecanismo que hace correcto al diseño.

7) CP-09: ¿En qué estado se encontró el reset?

La activación del reset vacía el FIFO y descarta los datos en vuelo. Que ese comportamiento sea correcto solo queda demostrado si el reset llega **encontrando datos dentro**: un reset sobre un FIFO vacío no ejercita la ruta de descarte, porque no hay nada que descartar.

El punto se muestrea en cada activación de reset, tomando la ocupación del modelo de referencia en ese instante, con los bins de la tabla X:

Bin	Criterio
while_empty	el modelo estaba vacío
while_partial	contenía entre 1 y Depth-1 elementos
while_full	contenía Depth elementos

TABLE X

BINS DE CP-09: OCUPACIÓN DEL FIFO EN EL INSTANTE DEL RESET.

Justificación: Es el punto que distingue un test de reset que realmente probó algo de uno que reinició un FIFO ocioso. Sin él, la contabilidad de descartes del scoreboard podría no haberse ejercitado nunca y el criterio de cierre no lo detectaría.

Nota de implementación. La ocupación que mide este punto es el tamaño del modelo de referencia, dato que reside en el **scoreboard** y no en el componente de cobertura. Se adopta la siguiente solución: al manejar el reset, el scoreboard **publica un evento por un puerto de análisis** con la ocupación descartada, y el componente de cobertura se suscribe a él.

Se descartaron dos alternativas: que la cobertura consulte directamente el modelo del scoreboard, que acoplaría dos componentes que deben ser independientes, y que el scoreboard muestree el covergroup, que trasladaría cobertura a un componente de comprobación.

La opción adoptada mantiene la cobertura puramente transaccional, igual que el resto de sus puntos, y deja además un registro explícito de cada reset con el número de elementos que se llevó, aprovechable en el capítulo de resultados.

8) CP-10: Bins por rango. ¿Cómo se consigue variar la relación?

Son *Bins* con periodos de reloj generados aleatoriamente, una relación exacta entre frecuencias es inalcanzable en la práctica. Los bins se definen por rango sobre el cociente rd_period / wr_period , en la tabla XI:

El punto se muestrea **una sola vez por corrida**, porque los generadores de reloj son estáticos y el periodo no cambia

Bin	Criterio	Interpretación
wr_faster	cociente > 1,1	el periodo de lectura es mayor
similar	$0,9 \leq \text{cociente} \leq 1,1$	relojes de frecuencia parecida
rd_faster	cociente < 0,9	el periodo de lectura es menor

TABLE XI

BINS DE CP-10: CLASES DE RELACIÓN DE FRECUENCIAS ENTRE LOS DOS RELOJES.

durante la simulación. Cerrar CP-10 exige por tanto variar la relación **entre corridas**, no dentro de una.

Se adopta la randomización de los periodos a partir de la semilla, pero **dirigida a una clase**: el test declara cuál de los tres bins quiere ejercitar y el resolutor produce una geometría que cae dentro de él. Una randomización libre dejaría al azar qué bin visita cada corrida, y el barrido de las tres clases volvería a ser probabilístico; con la clase como objetivo, la regresión recorre los tres de forma determinista.

```
// 8,0 .. 20,0 ns, en decimas
constraint c_range {
    wr_tenths inside {[80:200]};
    rd_tenths inside {[80:200]};
}

// pct = 100 * rd / wr
constraint c_ratio_class {
    (target == RATIO_RD_FASTER) ->
        100*rd_tenths <= 88*wr_tenths;
    (target == RATIO_SIMILAR) ->
        (100*rd_tenths >= 92*wr_tenths &&
         100*rd_tenths <= 108*wr_tenths);
    (target == RATIO_WR_FASTER) ->
        100*rd_tenths >= 112*wr_tenths;
}
```

Por qué décimas de nanosegundo y no un real.

La randomización con restricciones de SystemVerilog opera únicamente sobre tipos integrales: `rand real` no existe [3]. La relación entre periodos no puede por tanto expresarse como restricción sobre nanosegundos. Se randomizan décimas como enteros y se convierte al final, con lo que todas las restricciones quedan lineales sobre enteros, que es la forma que el resolutor trata bien.

Por qué los márgenes no coinciden con los cortes de los bins. CP-10 corta en 89/90 y en 110/111, mientras que las restricciones usan 88 y 112. La holgura es deliberada: el componente de cobertura calcula el cociente en coma flotante y lo **redondea** al asignarlo a un entero [3], mientras que aquí se resuelve en aritmética entera exacta. Son dos cálculos independientes del mismo número y en la frontera pueden discrepar en una unidad, lo que colocaría la corrida en un bin distinto del que el test pidió. El coste del margen es una franja estrecha de cocientes inalcanzable; la alternativa es un fallo intermitente e irreproducible.

La clase objetivo la fija cada test en su constructor y el argumento `+RATIO_CLASS` la sobrescribe, que es como el script de regresión barre las tres. Los periodos concretos siguen pudiendo fijarse por línea de comandos, para permitir

corridas dirigidas y reproducibles.

Sin este mecanismo CP-10 es inalcanzable. Con periodos fijos, una regresión de N semillas ejercita N veces la misma relación de frecuencias: se cubriría un bin de tres, y los cruces CP-12 a CP-15 (que son el núcleo del plan) quedarían permanentemente en un tercio.

9) *Los cruces:* ¿Por qué son alcanzables en todas las combinaciones?

Los cruces suscitan una objeción razonable: si el reloj de lectura es más rápido. ¿Cómo se va a llenar el FIFO? La respuesta es que **la relación de frecuencias no determina el caudal**: La frecuencia del reloj fija el **techo** de transacciones por unidad de tiempo. El caudal real lo fijan los **retardos que introduce el agente** entre transacciones.

Un lector con reloj rápido pero retardos largos tiene menos caudal que un escritor con reloj lento y sin retardos. La condición de lleno depende del caudal, no de la frecuencia, y por eso los campos `pre_drive_delay` y `post_drive_delay` de los items de driver resultan necesarios: **son el mecanismo que hace alcanzables los cruces desfavorables**. La tabla XII recoge cómo se alcanza cada combinación.

Combinación	Cómo se alcanza
lleno × escritura más rápida	espontáneamente, con estímulo aleatorio
lleno × lectura más rápida	con retardos largos en el agente de lectura
vacío × lectura más rápida	espontáneamente
vacío × escritura más rápida	con retardos largos en el agente de escritura

TABLE XII

COMBINACIONES DE LOS CRUCES Y MECANISMO POR EL QUE CADA UNA RESULTA ALCANZABLE.

Ninguna combinación es estructuralmente inalcanzable, pero **las desfavorables NO se alcanzan de forma fiable con estímulo puramente aleatorio**. El cierre de CP-12 a CP-15 depende de los tests dirigidos (llenar, vaciar) ejecutados bajo cada relación de frecuencias. Esta dependencia debe estar prevista en la planificación: la cobertura de los cruces no es un subproducto de la regresión aleatoria.

Importancia de los cruces: Alcanzar la condición de lleno es trivial, alcanzarla **bajo cada relación de frecuencias** es lo que ejercita realmente el cruce de dominios. Porque la latencia de sincronización se manifiesta de forma distinta según qué reloj sea más rápido. Un banco que solo llene el FIFO con relojes de frecuencia parecida deja sin verificar el caso en que el puntero remoto llega con varios ciclos de retraso, que es precisamente donde los reportes de ocupación pueden dejar de ser conservadores.

Razón de la simetría: Los cuatro cruces son los que resultan de aplicar el criterio anterior sin excepciones, y esa ausencia de excepciones tiene respaldo en el propio diseño. Las dos condiciones límite se calculan de forma simétrica,

cada una comparando el puntero local con el puntero remoto **sincronizado**:

```
assign full_wclk =
(fifo_wptr_q == (fifo_rptr_sync_q ^ xor_mask));
assign empty_rclk
= (fifo_wptr_sync_combi == fifo_rptr_q);
```

La latencia de sincronización se manifiesta igual en ambas, de modo que cualquier argumento que justifique cruzar la condición de lleno con la relación de frecuencias justifica igualmente cruzar la de vacío. Lo mismo vale para las dos vistas de la ocupación: CP-04 y CP-05 miden magnitudes distintas (una conservadora por exceso, la otra por defecto) y es precisamente bajo relaciones de frecuencia extremas donde esa diferencia se ensancha.

10) *Cobertura excluida y su justificación:*

- **Desfase en la liberación del reset:** PROP-12 establece que la liberación de ambos resets puede estar desfasada, lo que sugeriría un punto que distinguiese liberación simultánea de liberación desfasada. Se descarta por dos razones.

- 1) **El bin de liberación simultánea es estructuralmente inalcanzable:** Cada reset se libera en un flanco de su propio reloj, de modo que una liberación simultánea exigiría que ambos flancos cayesen en el mismo instante. Con periodos arbitrarios (y más aún randomizados) eso no ocurre. El punto quedaría con un bin imposible y el otro al 100 % desde la primera corrida, y **un bin que no puede dejar de llenarse no mide nada**.
- 2) **El caso que sí interesa no depende de la randomización:** Un desfase de muchos ciclos, con un dominio ya operativo mientras el otro sigue en reset, solo se produce si un test lo provoca deliberadamente. Es objeto del test dirigido de reset, y un punto de cobertura que únicamente se llena al ejecutar ese test no informa de nada que el listado de tests no diga ya.

- **Conmutación de los bits del camino de datos:** Se consideró un punto que verificase que cada bit de `wdata_i/rdata_o` toma los valores 0 y 1. Se descarta por dos razones:

- 1) **El diseño no interpreta los datos:** El camino de datos es una memoria. Por tanto no existe comportamiento que dependa del valor transportado, de modo que la cobertura de bits no distingue escenarios funcionalmente distintos.
- 2) **Es cobertura de conmutación, no funcional:** Las herramientas de cobertura de código la calculan automáticamente. Reproducirla como cobertura funcional duplica la métrica sin añadir información.

Un bit permanentemente fijo en el almacenamiento, además, lo detecta el scoreboard con el primer dato aleatorio que lo ejercite, sin necesidad de un punto de cobertura dedicado.

11) *Ubicación de la instrumentación:* El reparto sigue la distinción entre **cobertura de protocolo** y **cobertura**

funcional, que es el criterio del que nace la convención habitual de UVM, resumido en la tabla XIII:

Tipo	Qué mide	Dónde reside	Por qué
Protocolo	Variantes del handshake en una interfaz	Componente de cobertura de cada agente	Pertenece al protocolo, no a este diseño: viaja con el agente si se reutiliza
Funcional	Estados interesantes del DUT	Componente de cobertura del entorno	Pertenece a este diseño; a menudo exige correlacionar ambas interfaces

TABLE XIII

CRITERIO DE REPARTO ENTRE COBERTURA DE PROTOCOLO Y COBERTURA FUNCIONAL.

Aplicado a los puntos de este plan, la tabla XIV:

Componente	Puntos
prim_wr_coverage (agente de escritura)	CP-01, CP-03a
prim_rd_coverage (agente de lectura)	CP-02, CP-03b
prim_async_fifo_coverage (entorno)	CP-04, CP-05, CP-06, CP-07, CP-08, CP-09, CP-10, CP-12, CP-13, CP-14, CP-15

TABLE XIV

UBICACIÓN DE LA INSTRUMENTACIÓN: QUÉ PUNTOS RECOGE CADA COMPONENTE.

Observación sobre el desequilibrio: El reparto resulta muy asimétrico, y conviene explicar por qué en lugar de forzar una simetría artificial. De los diez puntos, ocho miden **estado del diseño** (ocupación, lleno, vacío, vuelta de punteros, reset) y solo dos miden (el protocolo).

La causa es la naturaleza del DUT: un FIFO no interpreta los datos ni ofrece variantes de operación, de modo que su protocolo de interfaz es mínimo (un handshake valid/ready de dos señales) mientras que su estado interno es lo verdaderamente interesante.

Es importante no confundir la **ubicación de la señal** con la **naturaleza del punto**: wdepth_o es una señal de la interfaz de escritura, pero lo que reporta es estado interno del FIFO. Un agente reutilizable para un protocolo valid/ready genérico cubriría patrones de handshake, no ocupación, porque la ocupación no existe en el protocolo: existe en este diseño.

Los componentes de cobertura se conectan al puerto de análisis del monitor correspondiente y se instancian únicamente si el objeto de configuración lo habilita, de modo que una regresión pueda desactivar la recolección sin recompilar.

E. Plan de pruebas

Las propiedades dicen **qué debe cumplirse siempre** y el plan de cobertura **qué situaciones hay que haber visitado**. Falta la tercera pieza: **qué tests producen esas situaciones**. Sin ella, la advertencia del apartado III-D.10 (que nada avisa si un escenario previsto no llega a escribirse) no tiene dónde compararse.

Los identificadores siguen el mismo criterio que PROP y CP: enunciado, justificación y trazabilidad. El estímulo se especifica como **restricciones sobre las secuencias**, nunca como código: un test dirigido restringe la aleatoriedad, no la sustituye. Fijar cada retardo a mano ejercitaría un único caso, mientras que restringir el caudal y dejar el resto aleatorio ejercita una familia entera y la explora con cada semilla.

1) *Catálogo de pruebas:* La tabla XV resume las siete entradas; los apartados siguientes detallan cada una.

Id	Nombre	Propósito	Tipo
TEST-01	Smoke	La infraestructura completa está viva	integración
TEST-02	Llenado	Alcanzar la condición de lleno	dirigido
TEST-03	Drenaje	Recorrer la ocupación de lleno a vacío	dirigido
TEST-04	Concurrencia a máximo caudal	Ambos dominios sin retardos	dirigido
TEST-05	Aleatorio con relación de frecuencias variable	Explorar bajo distintos ratios de reloj	aleatorio
TEST-06	Reset en vuelo	Reset con el bus y el FIFO ocupados	dirigido
TEST-08	Inyección de fallos (<i>familia, opcional</i>)	Validar que los comprobadores detectan	negativo

TABLE XV

CATÁLOGO DE PRUEBAS.

Qué significa aprobado. El veredicto de una corrida no se deduce de la ausencia de mensajes de error. El banco emite al terminar un **marcador positivo** que reúne las tres fuentes de fallo posibles (errores de UVM, fatales y fallos de aserción), y su ausencia se interpreta como fallo cualquiera que sea la causa; el mecanismo y las razones que lo imponen se detallan en el apartado VII-C. Ese veredicto se aplica a todas las entradas del catálogo, de modo que el campo *Aprobado* de cada una enuncia únicamente **la condición propia del test**, que se suma a él.

• TEST-01: Smoke.

- *Propósito:* Comprobar que reset, agentes, scoreboard, aserciones y cobertura funcionan en conjunto. No persigue esquinas.
- *Estímulo:* Ocho transferencias por dominio, retardos cortos, ambos agentes en paralelo.
- *Ejercita:* PROP-01, 02, 03, 04, 05, 07, 08.
- *Cierra:* CP-01, CP-02, CP-04, CP-06, CP-07, CP-08.
- *Aprobado:* El invariante del scoreboard cuadrado.

• TEST-02: Llenado.

- *Propósito:* llevar la ocupación hasta Depth y mantenerla.
- *Estímulo:* escritura con n_items == Depth y max_delay == 0; sin lecturas durante la fase de llenado, y drenaje posterior para no terminar con datos dentro.
- *Ejercita:* PROP-10 (wready_o inactiva con el FIFO lleno) y PROP-08. *Cierra:* CP-04 full, CP-06 reached.

- *Aprobado*: No debe dispararse el umbral de transacción atascada: si lo hace, el test pidió más escrituras de las que caben y el defecto está en el test.

- **TEST-03: Drenaje.**

- *Propósito*: Recorrer toda la ocupación observable desde el dominio de lectura.
- *Estímulo*: llenado como en TEST-02, seguido de Depth lecturas espaciadas (max_delay alto) y **sin escrituras concurrentes**.
- *Ejercita*: PROP-01, PROP-11, PROP-07. *Cierra*: **CP-05 completo** y CP-07 reached.
- *Aprobado*: El FIFO vacío al final.

- **TEST-04: Concurrencia a máximo caudal.**

- *Propósito*: someter el cruce de dominios a la máxima exigencia, con ambos punteros avanzando en ciclos consecutivos.
- *Estímulo*: los dos agentes con max_delay == 0 y n_items múltiplo de Depth, en paralelo.
- *Ejercita*: PROP-07 de forma sostenida y PROP-01 a PROP-03 sobre una secuencia larga. *Cierra*: CP-01 y CP-02 back_to_back, CP-08 con vueltas repetidas.
- *Aprobado*: Ninguna condición adicional al veredicto general.

- **TEST-05: Aleatorio con relación de frecuencias variable.**

- *Propósito*: Explorar el espacio de estímulo bajo relaciones de frecuencia distintas.
- *Estímulo*: Periodos de reloj derivados de la semilla; n_items y max_delay totalmente aleatorios en ambos dominios.
- *Ejercita*: Todas las propiedades, de forma no dirigida. *Cierra*: CP-10, y **es condición necesaria** para CP-12 a CP-15.
- *Aprobado*: Veredicto de aprobado en **todas** las semillas de la regresión, no en una cualquiera.

- **TEST-06: Reset en vuelo.**

- *Propósito*: Activar el reset cuando hay algo que descartar y alguien esperando.
- *Estímulo*: llenado parcial o total, y activación del reset **mientras una transferencia está pendiente** (valid afirmada y ready todavía baja), situación que se provoca con contrapresión.
- *Ejercita*: PROP-06a a 06d, PROP-12, PROP-13. *Cierra*: CP-03 during_transfer, CP-09 while_partial y while_full.
- *Aprobado*: El invariante lecturas + descartadas_por_reset == escrituras cuadrado.

2) *TEST-08: inyección de fallos (familia, opcional)*: **Estado: fuera del alcance comprometido**. La técnica se aplica y se evidencia en este trabajo: seis inyecciones manuales, una por comprobador, con sus resultados en el apartado VI. Lo que esta entrada describe es su **automatización** como tests

permanentes de regresión, que protege frente a regresiones a lo largo del mantenimiento pero no aporta un hallazgo nuevo. Se documenta con el mismo detalle que el resto para que el trabajo pendiente quede especificado, y se declara opcional por la misma razón por la que el apartado III-G declara fuera de alcance la verificación formal: el criterio no es lo interesante que sea una técnica, sino si cabe en el alcance.

- *Propósito*: Demostrar que cada comprobador detecta la violación que dice detectar.
- *Estímulo*: El de TEST-01, sobre una versión del entorno con **un** defecto introducido deliberadamente, uno por corrida.
- *Ejercita*: La infraestructura de comprobación, no el DUT.
- *Cierra*: Ningún punto de cobertura.
- *Aprobado*: **invertido**. Única entrada del catálogo que no hereda el veredicto general, sino que lo niega: el test pasa cuando el comprobador **sí** dispara, y con el identificador esperado. Una corrida limpia es un fallo de esta familia. Por eso el criterio se aísla en un método virtual del test base.

Catálogo de la familia: Un defecto por corrida, seleccionado por línea de comandos. Se inyecta de uno en uno porque con varios activos no se puede **atribuir** qué error procede de cuál, los defectos interactúan, y un comprobador averiado quedaría enmascarado por otro que sí dispara; y la atribución es justamente lo que esta familia demuestra. El catálogo es el de la tabla XVI.

Los dos mecanismos, y por qué son dos:

- **Mecanismo I**: Defecto en un componente UVM. Se instala por *type override* de factory, sustituyendo el monitor o el scoreboard por una variante defectuosa. El comprobador emite un informe de UVM con identificador propio, de modo que el test puede interceptarlo con un `uvm_report_catcher`, contarlo y degradarlo a informativo. Sin esa degradación la corrida terminaría con `UVM_ERROR > 0` y la regresión la marcaría como fallida, cuando ese error es precisamente el resultado buscado.
- **Mecanismo II**: Violación de una aserción. Las aserciones viven en las interfaces y emiten `$error`, que no pasa por el sistema de reporte de UVM. La decisión es deliberada (las interfaces deben seguir sirviendo en bancos sin UVM y en un eventual flujo formal) y su consecuencia aquí es que **un `uvm_report_catcher` no puede verlas**. La violación se hace observable con un contador de fallos en la propia interfaz, incrementado en el bloque de acción de cada aserción junto al `$error`; el test lo lee por la interfaz virtual en su `check_phase`. Un contador entero no obliga a la interfaz a importar nada.

TEST-08f dejó de ser alcanzable durante la construcción: La inyección manual correspondiente se ejecutó antes de que la interfaz modelara el reset del maestro. Con el gateado actual, descrito en el apartado IV, el agente **no**

Id	Defecto inyectado	Comprobador que debe disparar	Mecanismo
TEST-08a	El monitor de escritura publica el dato con un bit invertido	Scoreboard, discrepancia de valor (PROP-01)	I
TEST-08b	El monitor de lectura publica cada item dos veces	Scoreboard, extracción sobre modelo vacío (PROP-03)	I
TEST-08c	El monitor de escritura reporta <code>wdepth = 0</code>	PROP-07, lado escritura	I
TEST-08d	El monitor de lectura reporta <code>rdepth = Depth</code>	PROP-07, lado lectura	I
TEST-08e	El monitor de lectura descarta un item	Invariante de cierre del scoreboard (PROP-02)	I
TEST-08f	El driver conduce <code>wvalid_i</code> durante el reset	PROP-06a	inalcanzable
TEST-08g	La liberación del reset ocurre fuera de flanco	PROP-13	II
TEST-08h	El driver retira <code>wvalid_i</code> antes de recibir <code>wready_o</code>	PROP-04	II
TEST-08i	El driver cambia <code>wdata_i</code> con <code>wvalid_i</code> alta y <code>wready_o</code> baja	PROP-05	II

TABLE XVI

CATÁLOGO DE LA FAMILIA TEST-08: UN DEFECTO INYECTADO POR CORRIDA.

puede afirmar `wvalid_i` durante el reset por mucho que el driver lo intente: la propia interfaz lo impide. Violar PROP-06a exigiría un interruptor que desactivase ese gateado, es decir degradar el banco para poder probarlo. Se deja constancia porque es un resultado en sí mismo: el modelado que garantiza una propiedad es el que impide inyectar su violación.

Lo que esta técnica no alcanza, y por qué se declara: Las propiedades **PROP-06b**, **06c**, **06d**, **PROP-08**, **PROP-10** y **PROP-11** son aserciones sobre **salidas del DUT**. Violarlas no está al alcance del entorno de verificación: ni el driver ni el monitor pueden hacer que `wready_o` suba con el FIFO lleno. Exigiría un **DUT mutado** (una copia del RTL con un defecto introducido), que es una técnica distinta, con su propio coste de mantenimiento y su propio riesgo de que la copia se desincronice del original.

Queda fuera del alcance de esta familia, y se hace constar en lugar de omitirse: **seis de las catorce comprobaciones de protocolo no están validadas por inyección ni pueden estarlo con el mecanismo previsto**. Lo que sí puede afirmarse de ellas es que no están permanentemente calladas, porque todas dispararon durante la construcción del entorno (PROP-04, PROP-05 y PROP-06a lo hicieron contra defectos reales del driver) y que su condición de disparo es revisable

por inspección.

3) Trazabilidad inversa: ¿Qué test cierra cada punto?

Esta tabla es el propósito de la sección. Un punto de cobertura sin test asociado es un hueco de planificación, no de estímulo, y sin esta matriz no se distinguen.

En **negrita**, el test que cierra ese bin **de forma determinista, en toda corrida**. Otros tests pueden alcanzarlo por azar (un estímulo aleatorio suficientemente largo acaba visitando casi cualquier estado), pero esa cobertura **no es reclamable como cierre**: depende de la semilla, y por tanto puede desaparecer sin que nada haya cambiado en el diseño ni en el banco. La correspondencia completa está en la tabla XVII.

Punto	Cerrado por
CP-01, CP-02	TEST-01, TEST-04 (<i>garantiza el bin de espaciado nulo</i>)
CP-03	TEST-01 <i>during_idle</i> , TEST-06 <i>during_transfer</i>
CP-04	TEST-01, TEST-02
CP-05	TEST-03
CP-06	TEST-01, TEST-02
CP-07	TEST-01, TEST-03
CP-08	TEST-01, TEST-04
CP-09	TEST-01 <i>while_empty</i> , TEST-06 <i>while_partial</i> , <i>while_full</i>
CP-10	TEST-05
CP-12 a CP-15	TEST-02 , TEST-03 , TEST-04 : Nota: Ejecutados bajo cada relación de frecuencias, consolidados en regresión

TABLE XVII

TRAZABILIDAD INVERSA: QUÉ TEST CIERRA CADA PUNTO DE COBERTURA.

Ningún punto queda sin test asignado, y seis de las diez filas tienen más de un test, lo que da margen si alguno se recorta. Las entradas en **negrita** son las que **no admiten sustituto determinista**: son la lista exacta de tests que no pueden omitirse sin dejar el cierre del punto a merced de la semilla, y por tanto lo primero que debe protegerse si el alcance se reduce.

Consecuencia para el criterio de cierre: un bin alcanzado únicamente por el test aleatorio no cuenta como cerrado mientras no exista el test dirigido que le corresponde. De lo contrario el criterio mediría la suerte de una regresión concreta en lugar de la capacidad del banco, y un cambio de semillas lo invalidaría sin previo aviso.

La fila de los cruces es además la única que no se cierra con una corrida, por mucho que se ejecuten esos tres tests: la relación de frecuencias es constante dentro de cada simulación y el cierre solo existe tras consolidar varias.

4) *¿Qué no es un test dirigido?:* Tres prácticas quedan excluidas, porque producen cobertura sin producir verificación:

- **Forzar señales internas del DUT:** Para alcanzar un estado. El estado se alcanza conduciendo las interfaces;

si no es alcanzable así, no es alcanzable en el sistema real y el punto de cobertura está mal planteado.

- **Insertar esperas fijas:** Para sincronizar fases sin haber medido que hacen falta. Un retardo puesto por si acaso es una dependencia temporal que nadie sabrá justificar después y que se rompe en cuanto cambian los periodos de reloj.
- **Relajar la definición de un bin** para que un test lo alcance. Queda ya excluido por el criterio de cierre; se repite aquí porque es en la escritura de los tests donde aparece la tentación.

F. Criterio de cierre

La verificación se considerará completa cuando se cumplan simultáneamente:

- 1) **100 %** de cobertura sobre los puntos CP-01 a CP-10.
- 2) ≥ 90 % sobre los cruces CP-12 a CP-15, con cada bin no alcanzado justificado individualmente por inalcanzabilidad estructural.
- 3) Cero fallos en una regresión de **tres semillas por cada una de las tres relaciones de frecuencia**, para todos los tests no opcionales del catálogo.
- 4) Todas las propiedades PROP-01 a PROP-13 implementadas y activas durante la regresión.
- 5) Todos los tests **no opcionales** del catálogo de [III-E.1](#) escritos y en verde.

Qué cuenta como en verde, en los puntos tercero y quinto. No la ausencia de mensajes de error, sino la presencia del **marcador positivo** que el banco emite al terminar y que agrega errores de UVM, fatales y fallos de aserción. La distinción no es retórica: un registro truncado, un simulador que termina de forma anómala o un test que no llegó a arrancar producen todos un registro sin errores, y las aserciones SVA no incrementan el contador de UVM. El apartado [VII-C](#) detalla el mecanismo y el defecto real que obligó a adoptarlo.

Conviene precisar en qué sentido esto modifica el criterio. La formulación original de este apartado exigía **cero errores de UVM**, y esa comprobación resultó no ser suficiente para acreditar lo que el criterio pide. Sustituirla por el marcador positivo **endurece la verificación del cumplimiento**, no el umbral exigido: ninguna corrida que aprobase con el criterio nuevo habría fallado con el viejo, y al menos una que aprobaba con el viejo falla con el nuevo.

La familia TEST-08 no condiciona el cierre. Su propósito, demostrar que los comprobadores detectan la violación que dicen detectar, se cumple con las inyecciones manuales ya ejecutadas y documentadas; su automatización es trabajo de mantenimiento y no de verificación.

Sobre el tamaño de la regresión: el número se eligió por convergencia de los cruces, y al hacerlo apareció un matiz que conviene explicitar. **Lo que cierra los cruces no son las semillas, son las tres relaciones de frecuencia.** Como la relación es constante dentro de cada simulación, ninguna corrida puede superar un tercio en CP-12 a CP-15 por muchas semillas que se lancen: recorrer las tres clases es obligatorio, no una mejora marginal. Las semillas aportan

variedad de estímulo *dentro* de cada clase (retardos, datos, número de transacciones), y tres bastan para que la regresión cierre los sesenta y seis bins. Son, por tanto, 54 corridas.

Un bin no alcanzado se justifica documentando **por qué** es inalcanzable. No se admite como justificación relajar la definición del bin: modificar los bins para que se llenen invalida el criterio de cierre.

G. Fuera de alcance y justificación

1) *Resolución de metaestabilidad:* La simulación RTL opera exclusivamente sobre valores lógicos discretos y no modela niveles intermedios ni tiempos de asentamiento. En consecuencia, la corrección del sincronizador frente a la metaestabilidad es **inverificable por simulación**, con independencia de la calidad del testbench.

PROP-09 enuncia la **tolerancia del diseño a la incertidumbre de latencia** que la metaestabilidad provoca: que el FIFO se comporte correctamente tanto si el puntero cruza en el número mínimo de ciclos como si tarda más. Conviene no confundir enunciarla con verificarla: **esa propiedad tampoco se ejercita**, por razones que son las mismas y que se miden en el apartado [VI-E](#).

La verificación de la metaestabilidad corresponde a otras clases de herramienta:

- **Análisis estructural de CDC**, que comprueba que todo cruce dispone de sincronizador y que no existen rutas de convergencia peligrosas.
- **Análisis temporal estático**, que verifica el cumplimiento de los tiempos de setup y hold y permite calcular el tiempo medio entre fallos.

2) *Validación sistemática de los comprobadores:* La técnica se aplicó de forma manual y sus resultados constan en el apartado [VI](#), pero **su automatización como tests permanentes de regresión queda fuera por alcance temporal**. La familia TEST-08 del apartado [III-E.1](#) especifica el diseño; la recomendación de abordarla, con su justificación y sus límites, se expone en el apartado [VIII](#).

3) *Verificación formal:* Queda fuera por alcance temporal. Se señala que PROP-01 a PROP-03 son candidatas naturales a demostración formal, al ser propiedades de seguridad sobre un diseño de espacio de estados acotado.

4) *Síntesis e implementación física:* El trabajo se limita al nivel RTL.

H. Estrategia de verificación

Metodología: UVM 1.2 sobre SystemVerilog. **Arquitectura**, en líneas generales:

- **Dos agentes independientes**, uno por dominio de reloj, cada uno con su driver, sequencer y monitor. La separación es obligada: cada agente debe operar con su propio reloj.
- **Scoreboard** que recibe las transacciones de ambos monitores y mantiene un modelo de referencia. Dado que el comportamiento esperado es exactamente una cola, el modelo de referencia es una queue de SystemVerilog.

- **Aserciones SVA** integradas en las interfaces, para las propiedades de protocolo que se comprueban ciclo a ciclo.
- **Colector de cobertura** conectado a los monitores.

Generación de estímulo. Las secuencias randomizan tanto los datos como los intervalos entre transacciones. Este segundo aspecto es imprescindible: si escritor y lector operan al mismo ritmo, el FIFO nunca alcanza las condiciones de lleno ni de vacío, y buena parte del plan de cobertura resulta inalcanzable. La relación de frecuencias entre ambos relojes se randomiza igualmente a nivel de test.

El detalle de la arquitectura (nombres de clases, diagrama de bloques y conexiones TLM) no se anticipa aquí: pertenece al apartado [IV](#), escrito con el testbench ya construido. Este apartado se limita a las decisiones de arquitectura que **condicionan el plan**, que son las que debían tomarse antes de escribir una línea de código.

IV. ARQUITECTURA DEL TESTBENCH

A. Enfoque y orden de construcción

La figura [2](#) recoge la arquitectura resultante, a la que este capítulo llega paso a paso. En el capítulo anterior se establece, **qué** hay que verificar: trece propiedades, un plan de cobertura y un criterio de cierre. Este capítulo documenta **cómo** se construyó la infraestructura que las comprueba y **qué decisiones de diseño hubo que tomar y con qué evidencia se justificaron**.

El orden de construcción se definió pensando en que cada capa, modulo y clase debe poder probarse antes de que exista la siguiente, según las etapas de la tabla [XVIII](#).

Etapas	Contenido	Criterio de finalización
1	Banco plano en SystemVerilog, sin UVM	Un dato cruza el FIFO, visible en la onda
2	Interfaces, aserciones, items, paquetes, TB top	Las trece propiedades activas y comprobadas
3	Agentes: drivers, monitores, sequencers, secuencias	Tráfico de datos con las aserciones en silencio
4	Entorno, test base y primer test funcional	TEST-01 con 0 UVM_ERROR

TABLE XVIII

ORDEN DE CONSTRUCCIÓN DEL ENTORNO Y CRITERIO DE FINALIZACIÓN DE CADA ETAPA.

La decisión metodológica más relevante fue **escribir las aserciones de protocolo antes que los drivers**, invirtiendo el orden intuitivo. El apartado [IV-I](#) documenta el retorno concreto de esa decisión: **seis de los siete defectos del entorno no afectaban a la integridad de los datos**, de modo que una comparación de valores los habría dado por buenos; lo que fallaba era la forma de onda del protocolo, que es justamente lo que una aserción vigila.

B. Banco preliminar sin UVM

Antes de introducir UVM se construyó un testbench plano: dos generadores de reloj, un reset y un par de bloques *initial* conduciendo las señales del DUT directamente.

Cuando el primer test UVM falla, hay dos hipótesis posibles: el DUT no funciona, o el entorno está mal construido. Depurar ambas simultáneamente es considerablemente más costoso que descartar la primera de antemano. El testbench plano cierra esa hipótesis con un coste de pocas horas.

La validación del correcto funcionamiento del DUT de esta prueba se puede observar en la figura [1](#), donde se pueden observar los datos entrando por el dominio de escritura y saliendo por el de lectura.

C. La capa de interfaz

1) *Dos interfaces, una por dominio:* El DUT expone dos grupos de señales gobernados por relojes sin relación entre sí. Se implementaron dos interfaces independientes, `prim_async_fifo_wr_if` y `prim_async_fifo_rd_if`, cada una enganchada a su propio reloj.

La simetría es solo aparente. Ambos puertos son handshakes *valid/ready*, pero **con los papeles invertidos**, como recoge la tabla [XIX](#):

	Escritura	Lectura
Lo conduce el agente	wvalid_i, wdata_i	rready_i
Lo produce el DUT	wready_o, wdepth_o	rvalid_o, rdata_o, rdepth_o

TABLE XIX

REPARTO DE PAPELES EN LAS DOS INTERFACES: LA SIMETRÍA DE LOS HANDSHAKES ES SOLO APARENTE.

Esta inversión tiene consecuencias que reaparecen en toda la jerarquía de clases: el dato es **estímulo** en escritura y **respuesta** en lectura, y las propiedades de estabilidad del protocolo interrogan al agente en un lado y al DUT en el otro.

2) *Clocking blocks: el modelo de setup y hold:* Cada interfaz declara tres clocking blocks:

```
// CB implícito para SVA (Xsim no lo infiere)
default clocking cb @(posedge clk_wr_i);
endclocking
// CB para el driver
clocking drv_cb @(posedge clk_wr_i);
    default input #1step output #1ns;
    output wvalid_drv;
    output wdata_i;
    input wvalid_i;
    input wready_o;
    input wdepth_o;
endclocking
// CB para el monitor
clocking mon_cb @(posedge clk_wr_i);
    default input #1step;
    input wvalid_i;
    input wdata_i;
    input wready_o;
    input wdepth_o;
endclocking
```

Los **skews** modelan el comportamiento físico de un **flip-flop**. `input #1step` muestrea en la región *Preponed*

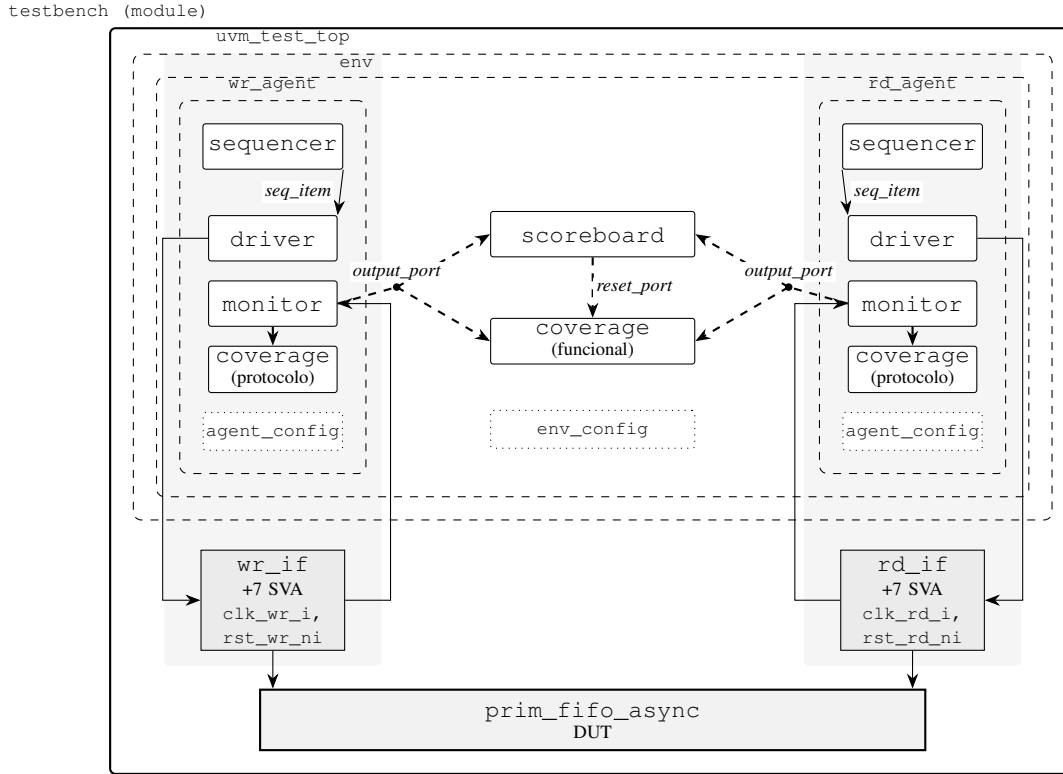


Fig. 2. Arquitectura del banco de pruebas. El sombreado agrupa los componentes de cada dominio de reloj; el scoreboard y la cobertura funcional son los únicos que reciben de los dos. Las flechas continuas son conexiones por señal y las discontinuas puertos de análisis TLM. Los objetos de configuración, en línea de puntos, se distribuyen por `uvm_config_db` y no por conexión estructural.

[3], es decir, el valor estable inmediatamente anterior al flanco: lo mismo que ve un flop en su ventana de setup. `output #1ns` conduce después del flanco, como un clock-to-Q. Esta disciplina elimina por construcción la carrera entre el testbench y el RTL, que en un simulador se despiertan en el mismo instante de tiempo.

La verificación experimental de esta semántica se documenta en el apartado VI, porque fue el origen de dos de los defectos encontrados.

3) *Modports: alcance real de la protección:* Se declararon tres modports por interfaz (dut, drv, mon) y los tipos de virtual interface correspondientes en `prim_async_fifo_types.sv`:

```
typedef virtual prim_async_fifo_wr_if
#(...) .drv prim_wr_drv_vif;
typedef virtual prim_async_fifo_wr_if
#(...) .mon prim_wr_mon_vif;
```

El agente guarda la interfaz completa y reparte vistas restringidas mediante `get_drv_vif()` y `get_mon_vif()`.

Se midió el alcance real de esta protección, con las tres pruebas aisladas de la tabla XX:

La tercera fila justifica la existencia de los modports: el clocking block por sí solo no protege, porque un componente con la interfaz completa puede saltárselo.

Debe señalarse una limitación: SystemVerilog permite asignar una vista restringida a un handle sin restringir,

Handle del monitor	Intento	Resultado
vista mon	conducir por <code>mon_cb</code>	rechazado en elaboración
vista mon	conducir la señal cruda	rechazado en elaboración
interfaz completa	conducir la señal cruda	permitido

TABLE XX

ALCANCE REAL DE LA PROTECCIÓN DEL MODPORT, MEDIDO CON TRES PRUEBAS AISLADAS.

recuperando el acceso sin necesidad de un *cast* explícito. El modport es por tanto una barandilla contra el descuido, no una barrera infranqueable. Protege contra el error de inercia (como copiar una línea del driver al monitor); no contra la intención deliberada.

4) *Modelado del reset asíncrono del maestro:* La propiedad PROP-06a exige que el agente no mantenga `wvalid_i` activa durante el reset. Al implementar el manejo de reset apareció un conflicto: el reset es asíncrono y llega a mitad de ciclo, mientras que un clocking block solo puede retirar una señal en el siguiente evento de reloj. En ese hueco la aserción fallaba, y lo hacía contra un DUT correcto.

Se evaluaron tres alternativas.

- **Alternativa A: Relajar la propiedad.** Admitir un ciclo de latencia en la retirada de `wvalid_i` y reescribir PROP-06a en consecuencia. Es defendible (modela un

maestro con reset síncrono en su flop de salida, que es un diseño legal) pero debilita la especificación para acomodar una limitación de implementación del banco, no del diseño.

- **Alternativa B: Conducir la señal directamente solo en el manejo de reset**, conservando el clocking block para el resto. Es la de menor coste aparente y la que se intenta primero, pero **no es viable**: una variable conducida por un clocking block no admite además asignación procedural, al ser dos conductores sobre la misma variable [3].

Lo que el estándar no fija, y sí se midió, es el **modo de fallo**: basta una asignación directa para que las conducciones posteriores a través del clocking block dejen de tener efecto durante el resto de la simulación, sin aviso y sin recuperación. Es el defecto 1 del apartado VI-B.

La elección queda por tanto en binaria: o se renuncia a los outputs del clocking block en todo el agente, perdiendo el modelado de clock-to-Q y la protección frente a carreras en **todas** las señales conducidas para resolver un problema que solo aparece en el reset, o se resuelve el reset por otra vía.

- **Alternativa C: Modelar el reset asíncrono dentro de la interfaz, que es la adoptada:**

```
// la conduce el driver por drv_cb
logic wvalid_drv = 0;
logic wvalid_i;
// reset asincrono del maestro
assign wvalid_i = rst_wr_ni ? wvalid_drv : 0;
```

El driver conduce su *petición* por el clocking block; el pin que ve el DUT la retira de forma combinacional en el instante del reset, igual que haría un flop con reset asíncrono en un maestro real. Con esta construcción PROP-06a se cumple literalmente y el driver mantiene la disciplina de clocking block sin excepciones, y PROP-06a se cumple literalmente sin relajarla.

Tiene además un valor descriptivo: el banco **modela explícitamente** el comportamiento de reset del maestro que se conectaría a este FIFO, en lugar de dejarlo implícito en la temporización del driver. El renombrado de la señal interna a wvalid_drv refuerza esa lectura. El driver ya no conduce el pin del protocolo, conduce una solicitud que el reset puede vetar.

Verificación. Con un reset asíncrono inyectado a mitad de transacción, fuera de flanco de reloj, la implementación previa producía dos violaciones de PROP-06a y la adoptada ninguna.

El **gateado por reset se aplica solo a wvalid_i**, y conviene distinguirlo de la separación entre variable conducida y pin, que se generalizó a las dos interfaces por un motivo distinto y se trata a continuación.

Un análisis de las trece propiedades muestra que ninguna restringe wdata_i ni rready_i durante el reset. En el caso de wdata_i, PROP-05 está deshabilitada durante el reset y además exige wvalid_i == 1, lo que ya no puede

suceder. En el de rready_i la razón es de fondo y no de conveniencia: PROP-06 lo trata como un **permiso**, no como una petición, y un consumidor que mantiene su permiso activo mientras el productor está en reset se comporta de forma legal. Gatearlo modelaría un maestro que este contrato no exige. Extender la técnica a esas señales habría añadido lógica sin valor de verificación y, en el caso de wdata_i, habría borrado de la onda el dato que se intentaba transmitir al abortar, que es información útil durante la depuración.

5) *Un objeto, un papel: la separación entre petición y pin*: La interfaz de lectura reproduce la misma estructura de dos señales, pero sin gateado:

```
// la conduce el driver por drv_cb
logic rready_drv = 0;
wire rready_i;
assign rready_i = rready_drv;
```

La asignación es un simple búfer, de modo que la separación no añade comportamiento alguno. Existe porque **el driver necesita conducir la señal y además leerla**, para detectar que su petición llegó al cable, y con un único objeto haciendo los dos papeles hay que declararlo inout en el clocking block. Entonces el bloque muestrea, con skew de entrada #1step, una variable que él mismo conduce con skew de salida #1ns.

Resultado experimental. Con esa construcción, mon_cb devolvía X en el instante del reset, y el efecto observable no fue un error sino una ausencia: **CP-03b no llegó a registrar nunca su bin during transfer** y se quedó estancado en el 50 %. El punto de cobertura actuó aquí como detector de un defecto que ninguna aserción habría señalado, porque una señal desconocida no viola ninguna propiedad: simplemente deshabilita los antecedentes que la mencionan.

De ahí el principio que gobierna las dos interfaces: **un objeto, un papel**. Lo que el agente pide y lo que el DUT recibe son dos señales distintas aunque su valor coincida, y esa distinción es la que permite que el lado de escritura interponga el reset entre ambas sin tocar la disciplina del driver.

6) *Aserciones de protocolo*: Cada interfaz aloja seis aserciones SVA que implementan las propiedades de protocolo III-C. Su contenido y justificación se documentan en el apartado V, junto con el resto de la estrategia de comprobación.

Aquí interesa únicamente su ubicación arquitectónica: **las aserciones viven en la interfaz, no en un módulo aparte ni en un bind**. La razón es que operan sobre las señales del protocolo en su propio dominio de reloj, y la interfaz es el único punto donde esas señales y su reloj coexisten de forma natural. Todas se controlan con un interruptor has_checks gobernado desde el objeto de configuración del agente, cuyo valor por defecto es **activo**.

D. Estructura de paquetes y tipos

1) Reparto en paquetes:

- **prim_async_fifo_common_pkg**: tipos, item base, config base, reset handler

- **prim_wr_async_fifo_pkg**: agente de escritura completo
- **prim_rd_async_fifo_pkg**: agente de lectura completo
- **prim_async_fifo_pkg**: environment, scoreboard, cobertura funcional y su objeto de configuración
- **prim_async_fifo_test_pkg**: test base y tests

La existencia del paquete común busca alinearse con los principios de UVM [4] de reusabilidad y modularidad, pero su necesidad no es de estilo sino de compilación. En SystemVerilog **una clase se identifica por su declaración, no por su estructura** [3]: incluir el mismo fichero en dos paquetes no produce una clase compartida sino **dos clases distintas que casualmente se llaman igual**. Y tanto `extends` como `implements` se ligan a una declaración concreta, de modo que una clase base o un contrato duplicados dejan de relacionar entre sí a sus derivados de cada paquete.

Cuatro tipos tienen derivados o implementadores en más de un paquete y por eso viven en el común: la clase base de los sequence items, la base de los objetos de configuración, la base de los componentes de cobertura y la *interface class* de manejo de reset, que implementan trece clases repartidas por todos los paquetes del entorno.

En cuanto a las interfaces, se compilan desde la lista de fuentes y no se incluyen en ningún paquete, puesto que una interface no puede declararse dentro de un package [3].

El mismo principio (una clase es su declaración) determina dónde viven los sufijos de `uvm_analysis_imp_decl`. Esa macro no declara un puerto sino **una clase**, de modo que declarar el sufijo `_wr` en el scoreboard y otra vez en el componente de cobertura, ambos incluidos en el paquete del entorno, constituye una redefinición de clase. El criterio adoptado es por número de suscriptores: los sufijos `_wr` y `_rd` los emplean dos componentes (scoreboard y cobertura reciben los mismos items de los mismos monitores) y se declaran una sola vez en la cabecera del paquete; una única declaración sirve a ambos porque la clase generada está parametrizada por el componente que la implementa. El sufijo `_reset`, con un solo suscriptor, se declara dentro del componente que lo usa.

2) Jerarquía de sequence items: .

```
uvm_sequence_item
  prim_item_base
    prim_wr_item_drv
    prim_wr_item_mon
    prim_rd_item_drv
    prim_rd_item_mon
```

A continuación se desglosa lo que aporta cada uno:

- **prim_item_base**: `data`, `convert2string()`
- **prim_wr_item_drv**: `pre_drive_delay`, `post_drive_delay` (`rand`, `soft`)
- **prim_wr_item_mon**: `length`, `prev_item_delay`, `wdepth`
- **prim_rd_item_drv**: `pre_drive_delay`, `post_drive_delay` (`rand`, `soft`)

- **prim_rd_item_mon**: `length`, `prev_item_delay`, `rdepth`

El criterio no es *qué componente lo usa*, sino *qué naturaleza tiene el dato*. Los retardos de conducción son cosas que **se piden** y carecen de sentido como observación; `length` y `prev_item_delay` son cosas que **se miden** y carecen de sentido como estímulo. La distinción importa: `length` no es el retardo pedido, sino el número de ciclos que la transferencia estuvo realmente bloqueada, que puede diferir por la contrapresión del DUT.

En **prim_rd_item_drv** el campo `data` se desactiva para randomización (`data.rand_mode(0)`) porque en el dominio de lectura es la respuesta del DUT, no un estímulo. Es la manifestación en la capa de transacciones de la asimetría descrita en IV-C.1.

3) *Configuración: la jerarquía de clases base*: Los dos objetos de configuración comparten toda su funcionalidad salvo el tipo de la virtual interface. Se resolvió con una clase base abstracta **parametrizada por tipo**:

```
virtual class prim_agent_config_base
  #(type VIF_T      = int,
    type DRV_VIF_T = int,
    type MON_VIF_T = int)
  extends uvm_component;

  protected VIF_T vif;
  virtual function DRV_VIF_T get_drv_vif();
    return vif;
  endfunction
  virtual function MON_VIF_T get_mon_vif();
    return vif;
  endfunction
```

Las clases concretas se especializan y quedan reducidas a su registro en la factory. La base se declara *virtual class* (abstracta, no instanciable) y no se registra en la factory, puesto que no hay nada que construir.

Se descartó explícitamente la factory como mecanismo para esta variación. La factory decide *qué clase se instancia* en tiempo de ejecución; no puede alterar el *tipo de un miembro*, que se resuelve en elaboración. Además, las virtual interfaces no forman jerarquía de tipos: `prim_wr_vif` y `prim_rd_vif` no tienen ancestro común al que sobrescribir.

La configuración se implementa como `uvm_component` y no como `uvm_object`. La justificación es concreta: permite un `run_phase` que vigila coherencia en tiempo de ejecución, detectando si alguien modifica `has_checks` directamente sobre la interfaz en lugar de usar el `setter`. Un `uvm_object` no participa en las fases y no podría hacerlo.

Un ancestro común por encima de la clase parametrizada. Al incorporarse la cobertura funcional apareció un componente del entorno que necesita configurarse igual que los de los agentes (habilitar o deshabilitar su recolección) pero que **no tiene interfaz virtual**. Heredar de la clase parametrizada no era viable: obligaría a instanciarla con los tipos `int` por defecto, y además su `start_of_simulation_phase` aborta la simulación si la virtual interface es nula.

La jerarquía se resolvió extrayendo una base anterior, no parametrizada, con las banderas que no dependen de ninguna interfaz. El criterio del reparto es comprobable y no estético: **una bandera baja a la clase de agente si y solo si su manipulación toca la interfaz virtual**. El reparto resultante es el de la tabla XXI.

Bandera	Dónde vive	Por qué
has_coverage	base común	solo la lee el componente que decide si recolecta
has_recording	base común	solo la lee el componente que decide si registra
has_checks	clase de agente	su <i>setter</i> propaga el valor a <i>vif.has_checks</i> para habilitar las aserciones

TABLE XXI

REPARTO DE LAS BANDERAS DE CONFIGURACIÓN ENTRE LA BASE COMÚN Y LA CLASE DE AGENTE.

De la base común cuelgan dos ramas: la clase parametrizada de los agentes y la configuración del entorno.

E. Los agentes

Cada agente instancia su objeto de configuración, un monitor (siempre) y, si está en modo activo, un sequencer y un driver. El monitor se construye fuera de la condición de actividad: un agente pasivo conserva su capacidad de observación, que es precisamente su razón de ser.

A ellos se suma un **componente de cobertura de protocolo**, condicionado por su propia bandera de configuración. La consecuencia es que un agente puede encontrarse en cuatro configuraciones distintas (activo o pasivo, con recolección o sin ella), y el manejador de reset debe tolerarlas todas.

1) *Driver*: El driver obtiene la vista restringida *drv_cb* del objeto de configuración y conduce exclusivamente a través de ella. La estructura de una transferencia de escritura es:

```
// Fase de espera
repeat (item.pre_drive_delay) @(vif.drv_cb);
// Fase de petición
vif.drv_cb.wdata_i <= item.data;
vif.drv_cb.wvalid_drv <= 1;
// Fase de handshake: esperar al flanco
// en que AMBOS esten altos
@(vif.drv_cb);
while (vif.drv_cb.wready_o != 1
      || vif.drv_cb.wvalid_i != 1) begin
    @(vif.drv_cb);
end
// Fase de reposo
vif.drive_idle();
repeat (item.post_drive_delay) @(vif.drv_cb);
```

La condición del bucle comprueba el **pin**, no la petición interna. La versión inicial asumía que la transferencia ocurría cuando *wready_o* estaba alta, sin verificar que *wvalid_i* hubiera llegado al cable; el defecto resultante se documenta en el apartado VI.

2) *Monitor*: El monitor recibe la vista *mon_cb*, en la que todas las señales son de entrada. Reconstruye cada transacción detectando el handshake, mide *length* y *prev_item_delay*, captura la ocupación reportada en el instante de la transferencia y publica el item por un *uvm_analysis_port*.

3) *Manejo de reset*: Los componentes que deben reaccionar al reset implementan una interface class común:

```
interface class prim_async_fifo_reset_handler;
    pure virtual function void
        handle_reset(uvm_phase phase);
endclass
```

Una interface class es contrato puro (sin datos ni implementación) y por eso puede vivir en el paquete común sin ninguna dependencia de tipos.

El agente, en su *run_phase*, espera la activación del reset, invoca *handle_reset()* sobre cada hijo y espera su liberación. Cada componente mata su proceso de conducción o recolección y deja las señales en reposo.

Las invocaciones van protegidas con guardas de nulo, porque driver, sequencer y cobertura son condicionales: un agente pasivo, o con la recolección deshabilitada, no los construye, y sin los guardas el primer reset produciría una desreferencia nula. El monitor no la lleva porque se construye siempre.

Limitación del simulador. La propagación del reset se implementa con llamadas explícitas a cada hijo en lugar del recorrido idiomático sobre *get_children()* con *\$cast* a la interface class, porque XSim no implementa *\$cast* a *interface class*. El código idiomático queda comentado en el fichero del agente para facilitar el porte a otro simulador.

4) *Componente de cobertura de protocolo*: Cada agente incorpora un componente que recoge la cobertura **del protocolo**, no la del diseño. El criterio del reparto es el del plan de cobertura: lo que mide variantes del *handshake* pertenece al protocolo y viajaría con el agente si este se reutilizase en otro proyecto; lo que mide estados del FIFO pertenece a este diseño y reside en el entorno.

Aplicado aquí, cada agente recoge dos puntos: el **espaciado entre transacciones** (CP-01 en escritura, CP-02 en lectura) y el **estado del bus en el instante del reset** (CP-03, recogido por ambos).

El componente se suscribe al puerto de análisis del monitor, de modo que su recolección es puramente transaccional, con una excepción que conviene declarar en lugar de disimular: **CP-03 no puede recogerse de una transacción**. Requiere conocer si había una transferencia pendiente en el instante en que llegó el reset, y ese instante no viaja por ningún puerto de análisis. Se resuelve implementando también aquí la *interface class* de manejo de reset y leyendo el clocking block del monitor:

```
virtual function void
handle_reset(uvm_phase phase);
    cover_reset.sample(vif.mon_cb.wvalid_i
                      && !vif.mon_cb.wready_o);
endfunction
```

La condición es la misma que define `write_is_blocked` en PROP-04 y PROP-05: *valid* afirmada y *ready* todavía baja. Tomar únicamente *valid* habría contado como transferencia en curso un ciclo en el que el traspaso ya se había completado.

El valor procede del clocking block y no de la señal cruda, de modo que lo que se registra es el muestreo del último flanco. Es lo correcto para la pregunta que el punto formula (si había una transferencia pendiente en el ciclo en que llegó el reset) y evita además la carrera de leer una señal en el mismo instante en que el reset asíncrono la está modificando.

5) *Una base común para los tres componentes de cobertura*: Los dos componentes de protocolo y el del entorno miden cosas distintas, pero los tres necesitan lo mismo: llevar una **contabilidad por bin en paralelo al covergroup**, volcarla a fichero y comprobar que no ha divergido de él. La necesidad y su justificación, que nacen de que SystemVerilog no permita enumerar los bins de un covergroup, se exponen en el apartado VII-D; aquí interesa dónde reside el código.

Reside en una clase base abstracta común a los tres, que ofrece cinco servicios: declarar un bin, registrar un acierto, resumir un punto de cobertura, volcar la tabla completa a CSV y contrastar el porcentaje derivado contra el que reporta el covergroup. Nada de eso depende de qué se esté midiendo, y duplicarlo en tres sitios habría significado tres oportunidades de que las tres copias divergieran.

La decisión tiene una consecuencia que conviene registrar porque costó una sesión de depuración. Un componente derivado volvió a declarar por su cuenta el diccionario de aciertos y dos de los métodos, ocultando los de la base. El resultado no fue un error de compilación sino un volcado vacío: el componente contabilizaba sobre su copia y la base volcaba la suya. **Lo detectó el propio autochequeo**, que comparó un porcentaje derivado de cero aciertos contra un covergroup que sí había muestreado. Una comprobación escrita para vigilar la clasificación de los valores acabó detectando un problema de herencia, que es el tipo de rendimiento que justifica el coste de mantenerla.

F. Testbench top y arranque de UVM

El módulo testbench es la frontera entre el mundo estático (módulos, interfaces, señales) y el mundo dinámico de las clases. Sus responsabilidades, dentro de un único bloque `initial`, son tres:

- 1) Publicación de las virtual interfaces en el `uvm_config_db`, antes de `run_test()` porque el `build_phase` que las recoge ocurre dentro de él.
- 2) Arranque **diferido** de los generadores de reloj, y conducción del reset de encendido.
- 3) Llamada a `run_test()`.

El testbench no decide la geometría temporal: la consume. Los periodos de reloj los fija el `build_phase` del test base, que los randomiza dentro de la clase de relación de frecuencias pedida y los publica en la base de configuración. El módulo espera un evento, los lee y oscila. Esta asignación de responsabilidades no es arbitraria: quien genera los relojes es la única fuente de verdad sobre cuáles

son, y el componente de cobertura clasifica CP-10 sobre el mismo valor publicado. Un valor por defecto en el módulo sería un segundo origen de verdad, y un fallo en la lectura produciría relojes a una frecuencia y cobertura clasificada sobre otra, sin que nada lo advirtiera. Por eso la lectura es obligatoria y su fallo es fatal.

El arranque de los relojes se difiere hasta que el test los ha fijado, mediante un `uvm_event` que el test dispara en su `build_phase`. Todas las fases de construcción de UVM ocurren en tiempo cero, de modo que diferir el arranque no cuesta un solo picosegundo de simulación: se comprobó que una corrida con periodos fijos termina en el mismo instante que antes del cambio.

La espera del evento debe ocurrir en un proceso de-sacoplado del bloque principal. Esperarlo en línea bloquearía el `initial` antes de llegar a `run_test()`, el test nunca se construiría y nadie dispararía el evento. El síntoma de ese bloqueo mutuo es una simulación que termina en tiempo cero sin resumen de UVM y con código de salida cero, es decir indistinguible de un éxito.

Los periodos, cuando se fuerzan desde la línea de comandos para reproducir una corrida concreta, se leen con `$value$plusargs` y no a través de UVM. La razón se documenta en el apartado IV-J.

G. Entorno y tests

El paquete de tests es independiente del paquete del entorno: el entorno aspira a ser reutilizable, los tests son específicos de este proyecto.

1) *Composición del entorno*: El `environment` instancia cinco elementos: los dos agentes, el scoreboard, el componente de cobertura funcional y su objeto de configuración, que condiciona la creación del anterior.

La cobertura del entorno se suscribe a los puertos de análisis de **ambos** monitores y, además, a un tercer puerto que publica el scoreboard. Ese tercero merece explicación porque no es un patrón habitual: el punto CP-09 mide la ocupación del FIFO en el instante del reset, y ese dato solo lo conoce el scoreboard, que es quien mantiene el modelo de referencia.

Se descartaron dos alternativas. Que la cobertura consultase directamente el modelo del scoreboard acoplaría dos componentes que deben poder existir el uno sin el otro. Que la cobertura llevase su propio contador de ocupación crearía **un segundo modelo del FIFO**: el día que discrepara del primero, el informe de cobertura contradiría al comprobador sin que nada indicase cuál de los dos es correcto. La solución adoptada, que el scoreboard publique la ocupación por un puerto de análisis justo antes de vaciar su modelo, mantiene una única fuente de verdad y conserva la recolección de cobertura puramente transaccional.

El orden de las llamadas importa, y por eso CP-09 se muestrea **al recibir el evento** y no dentro del manejador de reset del componente de cobertura. Si se leyera allí, el resultado dependería de si el vigilante de reset del entorno invocó antes al scoreboard o a la cobertura: un acoplamiento

por orden de llamada que alteraría un número de cobertura al mover dos líneas de sitio.

2) *Mecánica de los puntos de transición*: Dos puntos del plan, la condición de lleno y la de vacío, no cubren valores sino **cambios de valor**. Su implementación descansa en una propiedad del lenguaje que conviene hacer explícita: **un coverpoint con bins de transición mantiene por sí mismo el valor de la muestra anterior** [3]. No es necesario conservar el item previo; el covergroup compara cada muestra con la que recibió en la llamada anterior de esa misma instancia.

Esa memoria implícita tiene una consecuencia que el plan no previó: **sobrevive al reset**. Si el FIFO estaba lleno cuando llegó el reset, la siguiente escritura muestrea «no lleno» y el covergroup registra la transición como *abandoned*. Pero el FIFO no se recuperó del lleno: lo vació el reset. El bin existe precisamente para demostrar que la condición límite es transitoria y no terminal, y un acierto obtenido por reset vacía de contenido esa demostración.

La solución es un **valor centinela** que no pertenece al dominio de la condición. El manejador de reset muestrea un valor negativo, y la secuencia resultante no encaja en ningún bin declarado. Al no haber bin que lo recoja, el centinela no altera el porcentaje.

El mismo manejador reinicia los contadores de CP-08. La vuelta de los punteros no es observable desde las interfaces, de modo que se deduce contando transacciones aceptadas: el RTL incrementa el puntero exactamente una posición por transferencia aceptada, luego el contador es una réplica del puntero. **Y como el reset lleva los punteros a cero, el contador debe acompañarlos**; conservarlo desplazaría todas las vueltas posteriores y el punto dejaría de medir lo que declara medir durante el resto de la corrida.

3) *Los cruces y la restricción del covergroup único*: Los cuatro cruces del plan enfrentan cada condición del diseño contra la relación de frecuencias. Su implementación choca con una restricción del lenguaje: **un cruce solo puede formarse entre coverpoints declarados en el mismo covergroup** [3], porque la semántica es «ambos aciertan en la misma muestra» y esa noción solo existe dentro de una instancia.

La consecuencia práctica es que el coverpoint de la relación de frecuencias **aparece replicado** en cada covergroup que lo cruza. No es duplicación por descuido, y para que no lo parezca los umbrales se declaran una sola vez como parámetros locales y los bins los referencian por nombre.

El covergroup que reporta CP-10 de forma independiente se conserva pese a esas réplicas, por un motivo que no es evidente: se muestrea una vez al arrancar la simulación, con independencia del estímulo. Las copias internas solo se muestrean cuando hay transacciones, de modo que un test sin escrituras dejaría CP-10 en cero aparentando un hueco inexistente.

4) *El test base*: Concentra las responsabilidades comunes a todos los tests:

- **Precondición de estímulo**: Ningún test conduce transacciones antes de que el reset de encendido se libere. Esta espera vive en el test base y no en

cada test derivado, que solo sobrescribe un gancho `run_stimulus()`. La justificación es un defecto real documentado en el apartado VI.

- **Cota temporal**: `uvm_root::get().set_timeout()` garantiza que ninguna corrida pueda colgarse indefinidamente: cualquier espera que no se cumpla termina en error visible en lugar de en un bloqueo silencioso.
- **Puente de argumentos de línea de comandos**: Tratamiento especial de lectura y aserción de los plus argumentos. Esto está relacionado a una limitación del simulador mencionado en el apartado IV-J, y abarcado con mayor detalle en VII.
- **Geometría temporal de la corrida**: fija los periodos de reloj, randomizados dentro de la clase de relación de frecuencias que el test pida, y los publica antes de autorizar el arranque de los generadores. El mecanismo se describe en el apartado IV-F.
- **Veredicto de la corrida**: al terminar, el test base emite un marcador que indica si la corrida pasó, a partir de los errores de UVM y de los fallos de aserción. La necesidad de un marcador positivo, y no de la mera ausencia de errores, se justifica en el apartado VII.

Las dos últimas responsabilidades sitúan al test base como la única autoridad sobre la configuración de la corrida y sobre su resultado, que es la razón por la que ambas viven aquí y no repartidas entre el módulo de nivel superior y cada test derivado.

H. Resultado del primer test funcional

El test de humo ejercita ocho transferencias por cada dominio, con retardos randomizados en ambos lados para desbalancear los ritmos.

```
escrituras observadas: 8
{10568, 21249, 3092, 32135,
56848, 40790, 7672, 46048}
lecturas observadas: 8
{10568, 21249, 3092, 32135,
56848, 40790, 7672, 46048}
UVM_WARNING: 0
UVM_ERROR: 0
UVM_FATAL: 0
```

La figura 3 recoge la forma de onda correspondiente. Los ocho datos atraviesan el FIFO conservando el orden, las catorce comprobaciones de protocolo permanecen en silencio y la simulación termina sin errores. Se cumple el criterio de aprobado de TEST-01.

Adicionalmente se verificó el escenario de **reset asíncrono a mitad de transacción**, inyectado fuera de flanco de reloj: el manejo de reset aborta las transacciones en vuelo sin producir violaciones de protocolo, frente a las dos violaciones de PROP-06a que producía la implementación previa al modelado descrito en el apartado IV-C.4.

I. Defectos encontrados durante la construcción

Durante la construcción del entorno se detectaron siete defectos en el propio testbench. Se documentan en el apartado

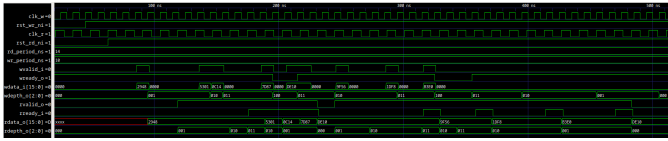


Fig. 3. Primer test conducido por la arquitectura de pruebas montada de UVM

VI, junto con el resto de resultados, porque constituyen evidencia del funcionamiento de la infraestructura de comprobación más que decisiones de arquitectura.

J. Limitaciones del simulador

Varias decisiones de implementación de este capítulo están condicionadas por limitaciones de XSim. Se recogen en el apartado VII, que las trata de forma unificada.

V. ESTRATEGIA DE COMPROBACIÓN

A. Reparto entre SVA y scoreboard

Las trece propiedades del apartado III no se comprueban con un único mecanismo. El reparto responde a una razón técnica y no de estilo, resumida en la tabla XXII:

Mecanismo	Qué comprueba	Por qué
SVA en la interfaz	Relaciones temporales acotadas sobre señales de un solo dominio	SVA es un formalismo síncrono: toda propiedad se evalúa sobre un reloj de muestreo
Scoreboard	Relaciones de datos sobre secuencias no acotadas que cruzan dominios	No existe un reloj común sobre el que escribir la aserción, y hace falta memoria de tamaño arbitrario

TABLE XXII

REPARTO ENTRE SVA Y SCOREBOARD, Y RAZÓN TÉCNICA DE CADA ASIGNACIÓN.

Para PROP-01, PROP-02 y PROP-03 (orden, ausencia de pérdidas y ausencia de duplicados) no hay reloj válido: los eventos de escritura y lectura viven en dominios sin relación entre sí, separados por una latencia que el propio DUT randomiza mediante `prim_cdc_rand_delay`. Una aserción necesitaría un reloj, y cualquier elección sería incorrecta. Exigirían además una cola de profundidad arbitraria, lo que equivale a escribir un scoreboard dentro de una aserción.

B. Aserciones de protocolo

Se implementaron **doce aserciones concurrentes**, seis por interfaz, correspondientes a las propiedades PROP-04, PROP-05, PROP-06 (subdividida), PROP-08, PROP-10 y PROP-11. A ellas se suman **dos comprobaciones inmediatas**, una por interfaz, para PROP-13, cuya naturaleza impide expresarla como aserción concurrente por el motivo que se expone en el apartado V-B.5. En total, catorce comprobaciones de protocolo sobre trece propiedades, recogidas en la tabla XXIII.

1) *Catálogo de aserciones:* Las catorce comprobaciones se recogen en la tabla XXIII.

Son las comprobaciones propias, no todas las que vigilan la corrida. El diseño verificado trae las suyas dentro: `prim_fifo_async` incluye, entre otras, `GrayWptr_A` y `GrayRptr_A`, que comprueban que ninguno de los dos punteros cambia más de un bit por transición, es decir la propiedad que sostiene el razonamiento del apartado II sobre el código Gray. Conviene inventariarlas antes de juzgar qué cubre un banco: darlas por inexistentes lleva a atribuirse huecos que no lo son, y el apartado VI-E documenta a dónde condujo no haberlo hecho.

La simetría entre las dos mitades no es completa, y las dos excepciones son informativas. PROP-06a recae sobre el **agente** mientras que su análoga en lectura, PROP-06c, recae sobre el DUT; el motivo se expone en el apartado V-B.3. Y PROP-10 y PROP-11 no son la misma propiedad vista desde cada lado, sino las dos condiciones límite opuestas, cada una observable únicamente desde su dominio.

2) *El interruptor de comprobaciones:* Todas las aserciones se controlan con un interruptor `has_checks` en la interfaz, gobernado desde el objeto de configuración del agente. Su valor por defecto es **activo**: la infraestructura de comprobación debe fallar en cerrado, de modo que cualquier ruta que no configure explícitamente el agente conserve las comprobaciones.

En las aserciones concurrentes el interruptor entra por la cláusula `disable iff` de cada `property`, y en las inmediatas como condición del bloque de acción. El apartado VI-B documenta el defecto que motivó fijar ese valor por defecto: una declaración sin inicializador dejó las doce aserciones desactivadas en silencio durante una fase entera del desarrollo.

3) *¿Cómo se subdividió PROP-06? El sujeto de una propiedad:* El enunciado de las cuatro subpropiedades figura en el apartado III-C y no se repite aquí. La subdivisión no formaba parte del plan original: es un refinamiento que impuso la implementación, y el episodio que lo provocó explica el reparto de sujetos que la tabla XXIII deja ver.

El enunciado de partida era «*no se produce ninguna transferencia mientras el reset está activo*». Al traducirlo a SVA resultó **imposible de escribir sin antes decidir algo que la propiedad no decía**: una transferencia es un handshake, el handshake tiene dos participantes, y una aserción necesita un sujeto. ¿Sobre quién recae la obligación, sobre el agente que afirma *valid* o sobre el DUT que responde *ready*?

La pregunta no es retórica, y la respuesta apareció al investigar un fallo de PROP-06a durante la escritura de las aserciones. Ninguna de las dos señales de respuesta del DUT está gateada por reset: son combinacionales sobre una comparación de punteros, y el reset deja los punteros iguales. Con punteros iguales, *vacío* es cierto y *lleno* es falso, de modo que **wready_o permanece alta durante todo el reset**: el diseño no se protege a sí mismo, y un agente que condujera `wvalid_i` en ese instante vería un handshake aparentemente completado mientras el puntero no avanza.

Etiqueta en el código	Prop.	Tipo	Qué exige
Interfaz de escritura			
PROP04_WVALID_STABLE_A	PROP-04	concurrente	wvalid_i no se retira antes de completar el handshake
PROP05_WDATA_STABLE_A	PROP-05	concurrente	wdata_i no cambia mientras la transferencia está bloqueada
PROP06A_NO_REQUEST_DURING_RESET_A	PROP-06a	concurrente	el agente no afirma wvalid_i con el reset activo
PROP06B_DEPTH_ZERO_DURING_RESET_A	PROP-06b	concurrente	wdepth_o vale cero con el reset activo
PROP08_WDEPTH_LEQ_DEPTH_A	PROP-08	concurrente	wdepth_o no excede Depth
PROP10_NO_READY_WHEN_FULL_A	PROP-10	concurrente	wready_o inactiva con el FIFO lleno
\$error("PROP-13...")	PROP-13	inmediata	rst_wr_ni se libera en flanco de clk_wr_i
Interfaz de lectura			
PROP04_RVALID_STABLE_A	PROP-04	concurrente	rvalid_o no se retira antes de completar el handshake
PROP05_RDATA_STABLE_A	PROP-05	concurrente	rdata_o no cambia mientras la lectura está bloqueada
PROP06C_NO_VALID_DURING_RESET_A	PROP-06c	concurrente	el DUT no afirma rvalid_o con el reset activo
PROP06D_DEPTH_ZERO_DURING_RESET_A	PROP-06d	concurrente	rdepth_o vale cero con el reset activo
PROP08_RDEPTH_LEQ_DEPTH_A	PROP-08	concurrente	rdepth_o no excede Depth
PROP11_NO_VALID_WHEN_EMPTY_A	PROP-11	concurrente	rvalid_o inactiva con el FIFO vacío
\$error("PROP-13...")	PROP-13	inmediata	rst_rd_ni se libera en flanco de clk_rd_i

TABLE XXIII

LAS CATORCE COMPROBACIONES DE PROTOCOLO. LAS ETIQUETAS SON LAS REALES DEL CÓDIGO Y PUEDEN LOCALIZARSE CON UNA BÚSQUEDA DE TEXTO SOBRE LAS FUENTES.

El manejo de reset del agente no cubre esa ventana, y conviene explicar por qué porque es la objeción inmediata. La **detección** del reset sí es instantánea, al ser asíncrona; la **retirada** de la señal no puede serlo, porque el driver conduce a través del clocking block y la asignación aterriza en el siguiente flanco de reloj. Entre un reset que llega a mitad de ciclo y ese flanco, wvalid_i sigue afirmada. La ventana dura menos de un ciclo y es suficiente: PROP-06a muestrea en ese flanco y dispara contra un diseño correcto.

Ese hallazgo fijó el reparto de sujetos. La obligación de no iniciar la transferencia en escritura **recae sobre el entorno de verificación y no sobre el DUT**, y es la única subpropiedad cuyo sujeto es el agente. De ahí procede también el modelado del reset asíncrono del maestro descrito en el apartado IV-C.4, que existe precisamente para garantizarla.

Del episodio se extrae una consideración metodológica, recogida entre las conclusiones del apartado VIII: una propiedad que no nombra a su sujeto parece completa en prosa y se revela incompleta en cuanto debe escribirse como aserción. El formalismo no introdujo la ambigüedad, la puso de manifiesto.

4) ¿Por qué las aserciones de reset no se deshabilitan con el reset?: Ocho de las doce aserciones concurrentes llevan el reset en su cláusula de deshabilitación, disable iff(!rst_ni || !has_checks), y las cuatro de PROP-06 no. La asimetría es deliberada y conviene justificarla, porque la línea uniforme es la que un vistazo rápido esperaría encontrar.

Una aserción **sobre el funcionamiento normal** debe deshabilitarse durante el reset: PROP-10 afirma que wready_o está inactiva con el FIFO lleno, y durante el reset esa afirmación no tiene sentido. Una aserción **sobre el reset** no puede deshabilitarse durante el reset, porque es su único momento de trabajo.

Llevado a PROP-06a, cuyo antecedente es precisamente rst_wr_ni == 0, añadir el reset a la cláusula la **anularía por completo**: con el reset inactivo el antecedente es falso y el intento tiene éxito de forma vacua; con el reset activo

la cláusula aborta el intento antes de evaluarlo. No queda ningún caso en que la propiedad afirme algo. La aserción no podría fallar bajo ningún estímulo, ni siquiera ante el defecto que existe para detectar.

El riesgo no es teórico sino **de copia**: la cláusula uniforme es la mayoritaria en el fichero, su forma es idiomática, y el resultado de extenderla compila, se lee bien y pasa en verde. Pertenece a la misma familia que el defecto 4 del apartado VI-B, comprobaciones desactivadas en silencio, con un agravante: aquí la inspección del código no basta, porque lo que está mal no es la línea sino su relación con el antecedente de la propiedad que gobierna.

Es también el caso concreto que justifica la **cobertura de aserciones** recomendada entre las líneas de trabajo futuro del apartado VIII. Un contador de fallos no distingue una propiedad vigilante de una que nunca llegó a evaluarse; un recuento de evaluaciones no vacuas, sí.

5) PROP-13: ¿Por qué no puede ser una aserción concurrente?: PROP-13 exige que el reset se libere en un flanco de reloj y nunca a mitad de ciclo. Una property con reloj **solo muestrea en los flancos** [3], de modo que es estructuralmente incapaz de observar una transición ocurrida entre ellos: preguntarle por lo que sucede a mitad de ciclo es pedirle justamente lo único que no ve.

La comprobación se implementa por tanto como aserción **inmediata**, disparada por el propio cambio del reset y contrastando ese instante contra el del último flanco:

```

realtime last_wr_clk_edge = 0;
always @(posedge clk_wr_i)
    last_wr_clk_edge = $realtime;

always @(posedge rst_wr_ni)
    if (has_checks &&
        $realtime != last_wr_clk_edge)
        $error("PROP-13: _rst_wr_ni_was
        _____deasserted_off_a_clk_wr_i_edge");

```

Una liberación mediante asignación no bloqueante tras @(posedge clk_wr_i) aterriza en la región NBA del

mismo instante, de modo que los tiempos coinciden y la comprobación pasa. Una liberación a mitad de ciclo no coincide y dispara. Ambos casos se verificaron experimentalmente.

El tipo de la variable que almacena el instante del último flanco no es indiferente. Debe ser `realtime` y no `time`: el segundo es un tipo integral, y almacenar en él el valor devuelto por `$realtime` redondea el instante [3]. Con periodos de reloj enteros el redondeo es invisible, pero con periodos randomizados los flancos caen en fracciones de nanosegundo y la comparación resulta falsa siempre, produciendo un falso positivo en cada liberación de reset. El defecto correspondiente se documenta en el apartado VI.

La corrección se garantiza además **por construcción**: la liberación del reset está encapsulada en la tarea `reset_deassert()` de la interfaz, que impone el flanco. Un test no puede violar PROP-13 aunque lo pretenda, salvo conduciendo la señal directamente.

C. Scoreboard y modelo de referencia

1) *El modelo*: El comportamiento esperado de un FIFO es exactamente una cola, de modo que el modelo de referencia es una `queue` de SystemVerilog y no requiere más estructura:

```
prim_async_fifo_data model[$];

// el monitor de escritura empuja
function void write_wr(prim_wr_item_mon t);
    model.push_back(t.data);
    ...
endfunction

// el de lectura extrae y compara
function void write_rd(prim_rd_item_mon t);
    // entrega sin escritura previa
    if (model.size() == 0)
        `uvm_error(...)
    esperado = model.pop_front();
    if (t.data != esperado)
        `uvm_error(...)
    ...
endfunction
```

Conviene señalar que esas pocas líneas cubren **las tres propiedades de integridad de datos a la vez**: el orden lo garantiza la disciplina FIFO de la cola, una pérdida aparece como discrepancia de valor, y un duplicado acaba produciendo una extracción sobre un modelo vacío.

2) *Recepción desde dos dominios*: ¿Por qué no hace falta sincronización?: La dificultad que cabía anticipar era el orden de llegada. Los dos monitores viven en dominios de reloj sin relación entre sí y publican de forma independiente, de modo que en principio nada garantiza que la transacción de escritura de un dato llegue al scoreboard antes que la de su lectura.

La causalidad lo resuelve sin mecanismo adicional. El diseño no puede entregar por el puerto de lectura un dato que no ha recibido por el de escritura, luego la publicación de la escritura precede necesariamente a la de su lectura. La cola no puede desordenarse.

Es una propiedad de este DUT y no un resultado general: un scoreboard que comparase dos flujos independientes sí necesitaría colas por flujo y una política de emparejamiento. Se documenta porque justifica la ausencia de una complejidad que cabría esperar encontrar.

3) *Dos puertos de análisis en un mismo componente*: Un `uvm_component` solo puede definir un método `write()` [4]. Para recibir de dos monitores se emplea la macro que genera variantes con sufijo:

```
`uvm_analysis_imp_decl(_wr)
`uvm_analysis_imp_decl(_rd)

uvm_analysis_imp_wr #(prim_wr_item_mon,
    prim_async_fifo_scoreboard) wr_imp;
uvm_analysis_imp_rd #(prim_rd_item_mon,
    prim_async_fifo_scoreboard) rd_imp;
```

La macro debe invocarse **una sola vez por sufijo y unidad de compilación**. El segundo parámetro de tipo es la propia clase: es así como el puerto sabe sobre qué objeto invocar `write_wr()` y `write_rd()`.

4) *Validación del scoreboard mediante inyección de fallos*: Un comprobador que nunca ha fallado no está verificado: puede ser correcto, o puede no estar conectado. Las dos ramas del scoreboard se validaron inyectando fallos en copias del entorno; los resultados, junto con los del resto de comprobadores, están en la tabla XXVII.

En el segundo caso el fallo se detectó por **dos caminos independientes**, la comprobación en tiempo de ejecución y la de fin de test, lo que confirma que ambas comprobaciones son independientes y no redundantes. Contra el diseño real, ninguna de las dos dispara.

D. Checker de ocupación: la propiedad CDC

1) *La comprobación*: ¿Por qué se se parte en dos mitades?: PROP-07 exige que se cumpla $rdepth_o \leq \text{ocupación real} \leq wdepth_o$. La ocupación real no es observable en ninguna señal: **es el tamaño del modelo de referencia**, y solo el scoreboard la conoce. Esa es la razón de que esta propiedad no pueda implementarse como aserción SVA.

La comprobación se reparte entre los dos puntos de recepción, cada uno verificando la mitad de la desigualdad que le corresponde:

```
// al recibir una escritura: el escritor
// nunca debe subestimar la ocupacion
if (model.size() > t.wdepth)
    `uvm_error("PROP07", ...)

// al recibir una lectura: el lector
// nunca debe sobreestimarla
if (model.size() < t.rdepth)
    `uvm_error("PROP07", ...)
```

Cada mitad detecta la dirección insegura de su dominio, y conviene no confundirla con la dirección permitida. Del lado de escritura la violación es que `wdepth_o` quede *por debajo* de la ocupación real: el escritor subestima lo que hay dentro, cree disponer de más hueco del que hay y puede desbordar. Del lado de lectura la violación es que `rdepth_o`

quede *por encima*: el lector sobreestima los datos disponibles y puede leer posiciones nunca escritas.

Las dos son igual de graves, y ninguna de las dos es la desviación que el diseño exhibe de forma legítima. Esa, la del apartado III-C —el escritor sobreestimando la ocupación y el lector subestimándola—, apunta al lado seguro y solo cuesta rendimiento. Lo que justifica comprobar las dos mitades por separado no es por tanto una diferencia de gravedad, sino de **atribución**: saber cuál de los dos reportes es el que miente.

Ambas direcciones se validaron por inyección, y el resultado consta en la tabla XXVII.

2) *Riesgo conocido: el desfase de publicación*: El tamaño del modelo no es la ocupación instantánea real sino la diferencia entre escrituras publicadas y lecturas publicadas, y ambos monitores publican con un ciclo de retraso en dominios distintos. Si una lectura ya ocurrió en el diseño pero su monitor aún no la ha publicado cuando llega una escritura, el modelo queda momentáneamente sobredimensionado y la comparación podría fallar sin defecto del diseño. No se ha observado en las campañas realizadas. Si apareciera, la corrección pasa por relajar la comparación con un margen igual al desfase máximo de publicación, no por eliminar la comprobación.

E. Comportamiento ante reset

El contrato está fijado en PROP-12: la activación del reset implica que el FIFO queda vacío, y los datos en vuelo se pierden legítimamente.

1) *Detección y propagación*: El scoreboard no observa señales físicas, solo recibe transacciones. Necesita por tanto que alguien le notifique el reset. Se evaluaron las cuatro ubicaciones de la tabla XXIV:

Alternativa	Objeción
El scoreboard obtiene su propia virtual interface	Da señales físicas a un componente de nivel de transacción
El monitor publica un ítem de reset	Mezcla eventos de control con transacciones en el mismo flujo
El agente notifica al scoreboard	El agente pasaría a conocer al scoreboard
El entorno coordina	Ninguna: es el único componente que conoce a ambos

TABLE XXIV

ALTERNATIVAS EVALUADAS PARA NOTIFICAR EL RESET AL SCOREBOARD, Y OBJECCIÓN A CADA UNA.

El entorno es el único que conoce a los agentes, que ya detectan el reset, y al scoreboard. Su `run_phase` ejecuta un observador permanente:

```
protected virtual task
reset_watcher(uvm_phase phase);
  forever begin
    fork
      wr_agent.wait_reset_start();
      rd_agent.wait_reset_start();
    join_any // basta uno: el modelo
              // ya no vale
    disable fork;
    scoreboard.handle_reset(phase);
```

```
if (coverage != null) begin
  coverage.handle_reset(phase);
end
fork
  wr_agent.wait_reset_end();
  rd_agent.wait_reset_end();
join // la liberación puede
      // estar desfasada
end
endtask
```

La asimetría entre los dos `fork` no es casual y refleja PROP-12 con precisión: `join_any` en la activación, porque la activación de **cualquiera** de los dos resets invalida el modelo; `join` en la liberación, porque el desfase entre liberaciones está soportado y debe esperarse a que ambos dominios estén operativos.

2) Manejar el reset frente a detectarlo:

El entorno **no** implementa la *interface class* `prim_async_fifo_reset_handler`. Driver, monitor, sequencer y scoreboard *manejan* el reset, una acción instantánea, de ahí que el contrato sea una *function*; el entorno *detecta y propaga*, lo que consume tiempo y exige una *task*. Son responsabilidades distintas y el lenguaje lo impone: una *task* no puede implementar una *function*.

3) *Contabilidad de los datos descartados*: Vaciar el modelo sin más produce un falso positivo: los contadores de escrituras y lecturas quedan descuadrados y la comprobación de fin de test reporta una pérdida que en realidad es un descarte legítimo.

La salvedad que PROP-02 ya recogía, «salvo reset intermedio», se hace contable:

```
virtual function void
handle_reset(uvm_phase phase);
  // publicación para la cobertura
  reset_port.write(model.size());

  reset_discarded_items_count +=
    model.size();
  model.delete();
  model = {};
endfunction
```

El invariante de cierre deja de ser «lecturas iguales a escrituras» para ser el que la propiedad realmente exige:

```
lecturas + descartadas_por_reset ==
    escrituras
```

Es una comprobación **más fuerte**, no más laxa: sigue detectando cualquier pérdida real y además distingue una pérdida de un descarte justificado. El informe final gana precisión, porque «8 escrituras, 6 lecturas, 2 descartadas» es una afirmación verificable mientras que «8 escrituras, 6 lecturas» no distingue un fallo de un reset.

4) *La ocupación en el instante del reset*: La primera línea de ese manejador escribe el tamaño del modelo en un puerto de análisis, antes de vaciarlo. No lo necesita el propio scoreboard: lo consume el componente de cobertura del entorno, cuyo punto CP-09 mide en qué estado se encontraba el FIFO cuando llegó el reset, vacío, parcialmente lleno o

llo. Sin ese dato, un reset sobre un FIFO vacío y uno sobre un FIFO llo serían indistinguibles en el informe de cobertura, y la contabilidad de descartes del apartado anterior podría no haberse ejercitado nunca sin que el criterio de cierre lo detectase.

La alternativa, que la cobertura llevase su propia cuenta de ocupación, se descartó por mantener **una única fuente de verdad**: dos modelos independientes del mismo estado pueden discrepar, y entonces el informe de cobertura contradiría al comprobador sin que nada indicase cuál de los dos se equivoca. La justificación completa de la decisión y de su orden de invocación está en el apartado **IV-G.1**.

5) *Verificación*: Escenario de reset asíncrono inyectado a mitad de transacción y fuera de flanco de reloj:

```
// TEST =
// prim_async_fifo_test_06_reset_in_flight
[SCOREBOARD] writes=18 reads=4
discarded_by_reset=14 left_in_model=0
UVM_ERROR: 0 UVM_WARNING: 0 UVM_FATAL: 0
```

Dieciocho escrituras, cuatro lecturas, catorce descartes por reset y ningún error: los dos datos perdidos en el reset se contabilizan como descarte legítimo. Sin el contador de descartes, el mismo escenario producía un error de lecturas no completadas.

Las catorce comprobaciones de protocolo permanecen en silencio durante todo el escenario, incluida PROP-06a, gracias al modelado del reset asíncrono del maestro descrito en el apartado **IV-C.4**.

F. Resumen del reparto de comprobaciones

La tabla **XXV** cierra el capítulo asignando a cada propiedad su mecanismo y el estado de su validación.

Sobre las marcadas como inalcanzables. No constituyen una omisión de planificación: son aserciones sobre **salidas del DUT**, y el entorno de verificación no puede violarlas. Ni el driver ni el monitor pueden hacer que `wready_o` suba con el FIFO llo, ni que `rdepth_o` cambie durante el reset. Validarlas exigiría un DUT mutado, que es una técnica distinta con su propio coste. La justificación completa y el catálogo de las que sí son alcanzables están en el apartado **III-E.1**.

Obsérvese la asimetría dentro de PROP-06: **06a es la única de las cuatro cuyo sujeto es el agente**, y es por eso la única inyectable. Es el mismo reparto de sujetos que motivó la subdivisión, visto ahora desde el otro lado.

VI. RESULTADOS

A. Primer test funcional

El primer test que ejercita el entorno completo conduce ocho transferencias por dominio con retardos aleatorios. Los ocho datos atraviesan el FIFO conservando el orden, las catorce comprobaciones de protocolo permanecen en silencio y la simulación termina sin errores.

Su valor no reside en lo que verifica, que es muy poco, sino en que **demuestra que la infraestructura entera está viva**: dos agentes conduciendo sus dominios, dos monitores

Propiedad	Mecanismo	Validada por inyección
PROP-01, 03: orden y duplicados	Scoreboard	Sí
PROP-02: ausencia de pérdidas	Scoreboard	No (<i>TEST-08e, opcional</i>)
PROP-04, 05: estabilidad del protocolo	SVA	No (<i>TEST-08h/i, opcional</i>)
PROP-06a: no iniciar transferencia en reset	SVA	Sí
PROP-06b a 06d: estado observable en reset	SVA	Inalcanzable
PROP-07: conservación de la ocupación	Scoreboard	Sí, ambas direcciones
PROP-08: cota superior de los reportes	SVA	Inalcanzable
PROP-09: tolerancia a la latencia variable	Implícita: el modelo no asume latencia	No ejercitada
PROP-10, 11: bloqueo por llo y por vacío	SVA	Inalcanzable
PROP-12: reset	Scoreboard	Sí
PROP-13: liberación síncrona del reset	SVA	Sí

TABLE XXV
RESUMEN DEL REPARTO DE COMPROBACIONES Y ESTADO DE SU VALIDACIÓN POR INYECCIÓN.

recolectando, el scoreboard comparando contra su modelo, las aserciones activas y la cobertura muestreando. Un fallo en cualquiera de esas piezas lo habría impedido, y a partir de este punto cualquier fallo posterior puede atribuirse a lo que se añadió después.

Con el scoreboard incorporado, el test cierra además su propia contabilidad: ocho escrituras, ocho lecturas, cero descartadas por reset y el modelo de referencia vacío al terminar.

B. Defectos encontrados en el entorno de verificación

Este apartado documenta los defectos detectados en el propio entorno de verificación. Se incluye porque ilustra el retorno de tres decisiones metodológicas: escribir las aserciones antes que los drivers, no dar por buena ninguna afirmación sin medirla, y randomizar los parámetros del entorno en lugar de fijarlos.

Ninguno de estos defectos afectaba a la integridad de los datos. En todos los casos el dato correcto llegaba al otro lado. Lo que fallaba era, en los seis primeros, la *forma de onda del protocolo*, que un scoreboard basado en comparación de datos no habría detectado; y en el séptimo, el comprobador mismo. La tabla XXVI los resume; los apartados siguientes comentan cada uno.

1) Defecto 1: conductores múltiples sobre una variable:

Una variable declarada como salida de un clocking block no admite además asignación directa. Se comprobó experimentalmente, **en XSim y sin poder contrastarlo en un segundo simulador**, que una única asignación directa **inutiliza el clocking block para el resto de la simulación**: tras ella, las conducciones posteriores a través del clocking block no tienen efecto. Esto descarta cualquier estrategia mixta y obliga a la regla «todo por clocking block, sin excepciones».

2) *Defecto 2: la misma causa, otro síntoma*: El driver mezclaba esperas sobre @(posedge clk) con señales gobernadas por el clocking block. Al no coincidir el instante en que el driver creía actuar con aquel en que la señal llegaba al cable, el monitor observaba **una transferencia donde había una sola** y publicaba el item por duplicado.

Comparte causa con el defecto 1 y difiere en el síntoma, lo que lo hace útil como contraste: la misma infracción de la disciplina del clocking block se manifiesta unas veces como bloqueo y otras como duplicado, de modo que el síntoma no permite deducir la causa. Lo detectaron **PROP-04 y PROP-05**, escritas días antes de que existiera el driver. Un scoreboard que solo compare datos no habría dicho nada: el dato duplicado era correcto.

3) *Defecto 3: el más instructivo*: El driver asignaba wvalid_i y comprobaba wready_o en el flanco inmediatamente siguiente. Pero las asignaciones del clocking block se aplican en el *siguiente* evento más el skew, de modo que en ese flanco la petición todavía no había llegado al cable. Con el FIFO vacío wready_o ya valía uno, el driver daba la transferencia por completada y retiraba la petición: el primer item de cada ráfaga se perdía sin ningún error. Los items siguientes no se veían afectados, porque el reposo del anterior y la petición del siguiente caían en el mismo

#	Síntoma	Causa	Detectado por
1	La simulación se bloquea tras la primera escritura	drive_idle() asignaba directamente señales que además conducía el clocking block	Traza de señales en los pines
2	El monitor reporta una transacción duplicada	Mezcla de @(posedge clk) con señales de clocking block en el driver	PROP-04 y PROP-05
3	Se pierde el primer item de cada ráfaga	El driver daba por hecha la transferencia sin verificar que wvalid_i hubiera llegado al cable	Conteo de handshakes en los pines
4	Las aserciones nunca fallan	bit has_checks; sin inicializador: valía cero y deshabilitaba las doce	Prueba negativa deliberada
5	wait_reset_end() retorna con el reset activo	La condición trataba el valor X como <i>reset liberado</i>	Traza del valor de reset en run_phase
6	Las secuencias terminan sin conducir nada	Carrera en t = 0 entre el run_phase del test y el del agente	Conteo de items recogidos
7	PROP-13 dispara en cada liberación de reset, en ambos dominios	time last_clk_edge es un tipo integral : redondeaba el instante del flanco	Primera corrida con periodos de reloj randomizados

TABLE XXVI

DEFECTOS DETECTADOS EN EL ENTORNO DE VERIFICACIÓN DURANTE SU CONSTRUCCIÓN.

evento de clocking, manteniendo wvalid_i alta de forma continua.

La detección requirió comparar *lo que el driver recibía del sequencer con los handshakes efectivamente ocurridos en los pines del DUT*. Ni los monitores ni una comparación de datos lo habrían revelado: ambos reportaban siete transferencias coherentes entre sí.

4) *Defecto 4: el más peligroso*: Un interruptor de comprobaciones sin inicializar dejó las doce aserciones desactivadas en silencio. La simulación pasaba en verde porque no se estaba comprobando nada. Se detectó únicamente porque se ejecutó una **prueba negativa deliberada**, esto es, forzar una violación conocida y verificar que la aserción efectivamente dispara. La práctica se adoptó como sistemática a partir de ese momento.

5) *Defecto 5: esperar un reset y darlo por liberado*: La tarea wait_reset_end() decidía con una comparación que trataba el valor X como **reset liberado**. Antes de que nadie condujera la señal, el reset vale X, de modo que la espera retornaba de inmediato y los componentes se daban por operativos con el reset todavía sin resolver.

Es el caso que fija el principio enunciado entre las lec-

ciones del apartado VIII: **una espera ante un valor desconocido debe esperar**. La corrección es una comparación de identidad, `! == 1` en lugar de la desigualdad lógica, que distingue «no es uno» de «es cero» y por tanto no concede la liberación a un valor que nadie ha establecido.

6) *Defecto 6: una carrera en el instante cero*: El `run_phase` del test y el de los agentes arrancan en el mismo instante de simulación, y nada ordena entre sí dos procesos que comienzan a la vez. Cuando el del test ganaba, las secuencias se ejecutaban antes de que el agente estuviera en disposición de conducir: **terminaban sin haber conducido nada**, y la corrida acababa sin errores porque tampoco había nada que comparar.

Se detectó contando los items recogidos por los monitores y contrastando esa cifra con la que las secuencias decían haber enviado. Es el mismo patrón que el defecto 4: **un fallo cuya única manifestación es la ausencia de trabajo**, invisible para cualquier comprobador que solo examine lo que sí ocurrió.

7) *Defecto 7: un falso positivo contra un diseño correcto*: La comprobación de PROP-13 contrasta el instante de la liberación del reset con el del último flanco de reloj:

```
// integral
time last_wr_clk_edge = 0;
// real
always @(posedge clk_wr_i)
    last_wr_clk_edge = $realtime;
```

`$realtime` devuelve un real y `time` es un tipo entero, de modo que la asignación **redondea** [3]. Con los periodos fijos del primer banco, 10 y 14 ns, todos los flancos caían en nanosegundos enteros y el redondeo resultaba invisible. Al randomizar los periodos los flancos pasaron a caer en fracciones, y la comparación resultó falsa siempre: la aserción denunciaba una liberación fuera de flanco que sí estaba en flanco.

Se distingue de los seis anteriores en dos aspectos. Es el primero que **acusa al diseño de un incumplimiento inexistente**, que es la clase de defecto más corrosiva de un banco de pruebas: erosiona la confianza en el propio comprobador, y la reacción instintiva, relajar la propiedad, destruiría justamente lo que la propiedad protege. Y es el primero que estuvo **latente durante fases enteras**, superando todas las regresiones, porque el estímulo carecía de la variabilidad necesaria para revelarlo.

Sobre cómo se encontró. La randomización de los periodos de reloj se introdujo para hacer alcanzable CP-10 y los cruces. Destapó este defecto en su **primera corrida**, antes de aportar un solo bin nuevo. Constituye el argumento empírico de por qué un estímulo con parámetros fijos es un riesgo y no solo un hueco de cobertura: no se sabe qué está ocultando hasta que se varía.

Sobre por qué estuvo a punto de pasar inadvertido. El resumen de UVM informaba de **cero errores**. Las aserciones emiten `$error`, que no incrementa el contador de `UVM_ERROR`, de modo que el fallo solo era visible leyendo el registro línea a línea. Es la misma carencia que motiva el marcador positivo de aprobado descrito en el apartado VII-C.

8) *Una característica compartida*: Cinco de los siete defectos coinciden en algo: el compilador los acepta y el fallo se manifiesta en otro lugar, o no se manifiesta en absoluto. El simulador no analiza el cuerpo de un método de clase hasta que algo la instancia, ni el de una `property` hasta que una aserción la invoca, ni advierte de la conversión implícita de un real a un tipo entero. Una declaración sin consumidor es código no verificado, y una conversión que el lenguaje permite no es necesariamente la que el autor pretendía.

C. Validación de la infraestructura de comprobación

Por la razón enunciada en el apartado V, un comprobador que nunca ha fallado no está verificado. Durante la construcción esta distinción resultó no ser teórica, pues las doce aserciones iniciales estuvieron desactivadas en silencio por un `bit` sin inicializar (defecto 4 de la tabla XXVI). Se adoptó por ello como práctica sistemática validar cada comprobador inyectando deliberadamente la violación que dice detectar.

La tabla XXVII recoge los resultados de las inyecciones realizadas, cada una sobre una copia del entorno:

Inyección	Comprobador	Errores
El monitor de escritura publica el dato con un bit invertido	Scoreboard: discrepancia de valor	8
El monitor de lectura publica cada item dos veces	Scoreboard: extracción sobre modelo vacío	8 + 1
El monitor de escritura reporta <code>wdepth = 0</code>	PROP-07, lado escritura	8
El monitor de lectura reporta <code>rdepth = Depth</code>	PROP-07, lado lectura	8
Liberación del reset 3 ns después del flanco	PROP-13	1
El driver conduce <code>wvalid_i</code> durante el reset	PROP-06a	2

TABLE XXVII

VALIDACIÓN DE LOS COMPROBADORES POR INYECCIÓN DE FALLOS. EN TODOS LOS CASOS SE ESPERABA UN NÚMERO DE ERRORES MAYOR QUE CERO.

Contra el diseño real, ninguno de estos comprobadores dispara. El conjunto discrimina por tanto entre comportamiento correcto e incorrecto, y no está ni permanentemente callado ni permanentemente activo.

Pendiente: su automatización. Estas inyecciones se realizaron de forma manual sobre copias del árbol de ficheros. Convertirlas en tests permanentes dentro del propio entorno, mediante *type override* de factory para las estáticas y *callbacks* para las que requieren activarse en un instante concreto, corresponde a la familia TEST-08 del apartado III-E.1, declarada opcional. Una inyección que solo existió una vez en una copia temporal no protege frente a regresiones.

D. Defectos encontrados en el DUT

No se ha encontrado ninguno.

`prim_fifo_async` es un diseño en producción de OpenTitan [5], revisado y ejercitado por otros bancos de pruebas durante años. Encontrar un defecto funcional en él

con un entorno construido en cuatro semanas habría sido notable, no normal. La ausencia de hallazgos no mide la calidad del banco: **mide que el DUT elegido era un buen patrón de referencia**, que es justamente lo que se buscaba al escoger un diseño ajeno y maduro en lugar de uno propio.

Lo que sí acredita este trabajo es que el entorno **habría detectado** los defectos que declara detectar. La evidencia no es la ausencia de errores sino la validación del apartado **VI-C**: cada comprobador se enfrentó a la violación que afirma vigilar y disparó, y contra el diseño real permanece en silencio. Un banco que no dispara nunca puede ser correcto o estar desconectado, y ambos casos son distinguibles porque se comprobaron los dos.

Qué habría hecho falta para encontrar algo. Las tres vías que este trabajo no recorrió, y que sí podrían revelar un defecto en un diseño de esta madurez, son la verificación formal de las propiedades de integridad, el análisis estructural de CDC con herramienta dedicada, y la exploración de geometrías distintas (otras profundidades y anchuras) que aquí quedaron fijadas en una sola configuración.

E. Un comprobador que disparaba sin que el veredicto lo supiera

El plan atribuía a PROP-09 la tolerancia del diseño a una latencia de cruce variable, y la daba por ejercitada porque el sincronizador de OpenTitan instancia `prim_cdc_rand_delay`. Al revisar esa justificación contra el RTL resultó que el módulo vive dentro de un ``ifdef SIMULATION` que este flujo **no define**, y que ni siquiera figuraba entre las fuentes compiladas: la latencia del cruce fue constante durante toda la campaña.

Conviene decir cómo apareció, porque forma parte del resultado. **No lo detectó el banco**: lo detectó releer el enunciado de una propiedad contra el código que supuestamente la sostiene. Y el banco no podía detectarlo, por la razón que el apartado **VIII** recoge como lección: una propiedad cuya comprobación se declara *implícita* no tiene comprobador cuyo silencio resulte sospechoso, ni punto de cobertura cuyo bin quede vacío.

1) El modelo tiene un envoltente, y la campaña lo excede: Incorporar el módulo y abrir sus dos puertas —el `define` y el `plusarg cdc_instrumentation_enabled`— deja la instrumentación operativa: mezcla bits en 122 de 200 transiciones, frente a ninguna sin el `plusarg`. Con ella activa, 36 de las 54 corridas fallan.

Esos fallos no acusan al diseño. El módulo compone su salida bit a bit entre el origen y `prev_data_i`, y el sincronizador conecta a ese puerto `intq`, el contenido actual del primer flop, no el valor que el cable tenía inmediatamente antes. Medido en la onda, con `wr = 12,7 ns` y `rd = 18,4 ns`: en `t = 209550 ps` el origen vale 010, el flop retiene 001 —dos pasos atrás— y la salida es 000, un valor que nunca estuvo en el cable.

No es un defecto del módulo: es uso fuera de su especificación. Su propia cabecera fija la condición, «*the delay should cause the input to be skipped by at most a single cycle*», y este trabajo la incumple a propósito, porque cerrar

CP-12 a CP-15 exige barrer relaciones de frecuencia de 0,4 a 2,5. De ahí que la regresión general no active la instrumentación y que ejercitar PROP-09 requiera un test aparte, acotado a ese envoltente.

2) Qué ocurre ante un defecto CDC real: Queda la pregunta pertinente, que no es si el diseño pasa sino si el banco **distinguiría un diseño incorrecto**. Se midió con una copia del DUT en la que `dec2gray` y `gray2dec` se sustituyen por identidades, de modo que el puntero cruza en binario: el defecto CDC de libro.

Sin instrumentación	Veredicto	Aserciones disparadas
DUT real, TEST-01 y TEST-04, dos clases	PASSED	0
DUT en binario, TEST-01	PASSED	8
DUT en binario, TEST-04	PASSED	128

TABLE XXVIII

EL DEFECTO SE DETECTA DE INMEDIATO Y EL VEREDICTO NO SE ENTERA.

El defecto **se detecta**, masivamente y sin necesidad de instrumentación alguna. Lo cazan `GrayWptr_A` y `GrayRptr_A`, dos aserciones que el propio `prim_fifo_async` lleva dentro y que comprueban que los punteros no cambian más de un bit por transición.

Y sin embargo el banco emitía ***** TEST PASSED ***** en los tres casos. El marcador positivo del apartado **VII-C** agrega errores de UVM, fatales y fallos de las aserciones *de las interfaces propias*; las del DUT emiten por el simulador sin pasar por ningún contador del entorno, de modo que quedaban fuera de la cuenta.

Es la tercera aparición del mismo patrón en este trabajo, tras el interruptor sin inicializar y el falso positivo de PROP-13: un comprobador que funciona, dispara y no llega al veredicto.

La corrección, sin embargo, no puede ser la misma. Esas aserciones **no forman parte del plan**: no se derivaron del contrato, no figuran entre las trece propiedades ni en la matriz de trazabilidad, y hacerlas condición de aprobado introduciría en el criterio de cierre algo que el criterio no declara. Tampoco conviene ignorarlas, porque son lo único que distingue un cruce correcto de uno roto. El guion de regresión **las cuenta y las avisa** sin que decidan el veredicto:

```
PASS prim..._test_04_concurrent similar seed=1
WARN aserciones propias del DUT: 128
      (GrayRptr_A GrayWptr_A)
```

La cifra es la que informa. Un disparo suelto tiene explicación benigna y aparece en la campaña: el reset asíncrono limpia el puntero a mitad de ciclo y se libera en el flanco siguiente, como PROP-13 exige, y en ese flanco el `disable iff` de la aserción ya no protege, de modo que compara el puntero recién puesto a cero contra su valor anterior. Ciento veintiocho disparos son otra cosa.

3) Qué se concluye: El diseño verificado tiene defensas propias, y la primera lección es que **conviene inventariarlas antes de medir lo que cubre el banco**: dar por sentado que las únicas comprobaciones son las que uno escribió lleva

a atribuirse huecos que no existen, y a no advertir que el veredicto estaba ignorando una fuente de error entera.

Sobre PROP-09, la conclusión es más modesta y queda en el apartado **III-D.1**: su enunciado es legítimo, el comprobador lo tolera por no modelar latencia, y ejercitarla exige el test acotado que la instrumentación permite construir dentro de su envolvente.

F. Regresión multi-semilla

La regresión final ejecuta los seis tests no opcionales bajo las tres relaciones de frecuencia con tres semillas cada una: **54 corridas, cero fallidas**. El script que la lanza construye un único *snapshot* y lo reutiliza en todas las ejecuciones, de modo que el coste agregado es el de una compilación más 54 simulaciones, y no el de 54 compilaciones.

El veredicto de cada corrida no se deduce de la ausencia de errores sino de un **marcador positivo** que el propio banco emite al terminar, y cuya ausencia se interpreta como fallo sea cual sea la causa. El motivo y su validación se exponen en el apartado **VII-C**.

Conviene dejar anotada una recomendación para cualquier fallo futuro: guardar siempre el número de semilla. Un fallo reproducible con una semilla conocida es un fallo que puede arreglarse; uno sin ella es una anécdota. El script imprime la combinación completa (test, clase de frecuencia y semilla) de cada corrida fallida, precisamente para que esa información no dependa de que alguien se acuerde de anotarla.

1) *Los tests dirigidos no eran deterministas*: El plan atribuye a cada test dirigido la responsabilidad de cerrar ciertos bins **en toda corrida**, y esa afirmación es a su vez una hipótesis que había que medir. Un barrido de 45 corridas, las tres clases de frecuencia por quince semillas, mostró que **TEST-02 y TEST-03 no la cumplían**. Las dos causas eran distintas y ninguna producía un fallo: las corridas terminaban en verde sin cubrir lo que decían cubrir.

Primera causa: fases no exclusivas. Ambos tests conducían los dos dominios a la vez, y sobre el estado del FIFO `max_delay` solo fija una **cota**, no un ritmo. Una semilla puede sortear para el lector un retardo menor que el del escritor y, bajo relojes con lectura más rápida, el FIFO no llega a llenarse en toda la corrida. Medido: **CP-06 caía al 0 % en una de cada quince corridas**. El test de llenado no llenaba.

La corrección consiste en dividir el estímulo en **fases exclusivas** en las que nunca conducen los dos dominios a la vez. Llevar el FIFO de vacío a lleno sin nadie leyendo no depende de la relación de relojes; hacerlo con un lector activo depende del sorteo. La concurrencia deliberada no desaparece del plan, se concentra en TEST-04, que es el test que la persigue.

Segunda causa: la latencia del cruce de dominios. Con las fases ya separadas, el llenado posterior al vaciado seguía sin registrar su transición. La razón está en PROP-07: `wdepth_o` es conservador por exceso porque el puntero de lectura llega sincronizado con retardo. Si la fase de rellenado arranca antes de que los vaciados hayan cruzado, la primera escritura encuentra `wdepth = Depth-1`, la

acepta y vuelve a dejarlo en `Depth`: la transición observada es **de lleno a lleno**, no cae en ningún bin, y `abandoned` no se registra. Medido: **CP-06 al 50 % en tres de cada cinco semillas** con lectura más rápida.

La corrección aquí no es esperar un número fijo de ciclos, que sería la misma carrera disfrazada de constante, sino **esperar a la condición, y observarla desde el dominio que debe verla**:

```
// hasta que el dominio de ESCRITURA
// vea el FIFO vacío
while (env.wr_agent.vif.wdepth_o != 0)
  @(posedge env.wr_agent.vif.clk_wr_i);
```

Lo que este episodio deja establecido es que una regresión en verde no acredita nada sobre la cobertura. Ninguno de los dos defectos produjo un error, y ambos eran invisibles en el porcentaje global, que seguía subiendo a base de corridas afortunadas. Se hicieron visibles porque la contabilidad por bin del apartado **VI-G** permite preguntar por *un* bin concreto en *una* corrida concreta, y porque la pregunta se formuló sobre 45 corridas en lugar de sobre una.

2) *Cierre determinista frente a cierre probabilístico*: El test aleatorio se ejecutó con entre 1500 y 2000 transacciones por corrida, volumen suficiente para que el estímulo visite la práctica totalidad de los estados del FIFO. El resultado sobre cinco semillas es el de la tabla **XXIX**:

Semilla	rd/wr	Escr.	CP-04	CP-05	CP-06	CP-07
1	1,238	1766	100%	75%	100%	100%
2	1,060	2016	100%	75%	100%	100%
3	0,652	1889	100%	75%	100%	100%
4	1,612	1574	100%	100%	100%	50%
5	1,218	1623	100%	100%	100%	100%

TABLE XXIX

COBERTURA ALCANZADA POR EL TEST ALEATORIO EN CINCO SEMILLAS, EN TANTO POR CIENTO.

Tres de las cinco semillas no cierran CP-05, y una tampoco CP-07. Una sola las cierra todas. El volumen de estímulo no compensa la ausencia de dirección: la cobertura aleatoria es **probabilística**, y un bin alcanzado en una corrida puede faltar en la siguiente sin que nada haya cambiado en el diseño ni en el banco.

De ahí el criterio adoptado en el apartado **III-E.3**: un bin alcanzado únicamente por el test aleatorio no se contabiliza como cerrado. Si se contabilizase, el criterio de cierre mediría la suerte de una regresión concreta, y un cambio de semillas lo invalidaría sin aviso.

Lo que el volumen no altera en absoluto. Los puntos dependientes del reset y de la relación de frecuencias permanecen **idénticos en las cinco corridas**: CP-09 y CP-10 en un tercio, y los cruces CP-12 a CP-15 también. Ni una transacción adicional los mueve, porque no son alcanzables por cantidad de estímulo sino por control explícito de dos condiciones del entorno. Es la confirmación empírica del razonamiento del apartado **III-D.10**.

Coste. Una corrida del test aleatorio genera 133 μ s de tiempo simulado y un volcado de 3,1 MB, frente a los 538 ns

del primer test funcional. La diferencia es de cinco órdenes de magnitud, y constituye un argumento adicional para que el cierre descansa en los tests dirigidos: además de fiable, es barato.

G. Cobertura final

La regresión completa ejecuta los seis tests no opcionales bajo las **tres relaciones de frecuencia** y con tres semillas cada una. El consolidado suma aciertos por bin y recalcula el porcentaje, sin promediar porcentajes entre corridas. El resultado consolidado es el de la tabla XXX.

Punto	Bins	Cob.	Aciertos
CP-01 Espaciado entre escrituras	3	100 %	18 471
CP-02 Espaciado entre lecturas	3	100 %	18 382
CP-03a Estado del bus al reset (escri.)	2	100 %	81
CP-03b Estado del bus al reset (lect.)	2	100 %	81
CP-04 Ocupación vista por escritura	4	100 %	18 471
CP-05 Ocupación vista por lectura	4	100 %	18 382
CP-06 Condición de lleno	2	100 %	1 001
CP-07 Condición de vacío	2	100 %	4 014
CP-08 Vuelta de punteros	2	100 %	9 204
CP-09 Ocupación en el reset	3	100 %	81
CP-10 Relación de frecuencias	3	100 %	54
CP-12 Cruce ocupación escri. × frec.	12	100 %	12/12
CP-13 Cruce ocupación lect. × frec.	12	100 %	12/12
CP-14 Cruce lleno × frec.	6	100 %	6/6
CP-15 Cruce vacío × frec.	6	100 %	6/6
Total	66	100 %	66 de 66

TABLE XXX
COBERTURA CONSOLIDADA SOBRE 54 CORRIDAS.

Sin agujeros: los sesenta y seis bins declarados tienen al menos un acierto, y las 54 corridas terminan con veredicto de aprobado. Con ello se cumplen los cinco puntos del criterio de cierre del apartado III-F.

Sobre los cuatro cruces. Ninguna corrida individual puede superar un tercio en ellos, porque la relación de frecuencias es constante durante toda una simulación. Su cobertura no es el resultado de un test sino de la regresión: existe únicamente tras consolidar corridas de las tres clases. Es la razón por la que el consolidado suma aciertos en lugar de promediar porcentajes, y la razón por la que el barrido por clase forma parte del script de regresión y no constituye una opción.

Sobre el recuento de bins. Los 66 comprenden los 30 de los once puntos de valor y transición, y los 36 de los cuatro cruces. Dos bins que el plan había previsto no existen, y su ausencia se justifica en el apartado III-D.1: `empty` en CP-04 y `full` en CP-05 son **estructuralmente inalcanzables**, porque ningún dominio puede observar el extremo del que él mismo acaba de alejarse.

H. Matriz de trazabilidad final

Este apartado cierra el círculo que abrió el plan de verificación: allí se declaró **qué** debe verificarse, y aquí se acredita **dónde está comprobado y con qué evidencia**. Las dos tablas se construyen a partir del código, de modo que pueden localizarse con una búsqueda de texto sobre las fuentes.

1) *Propiedades:* Tabla XXXI.

Recuento: catorce comprobaciones de protocolo, esto es, doce aserciones concurrentes (seis por interfaz) más dos comprobaciones inmediatas, sobre trece propiedades, de las cuales cinco se comprueban en el scoreboard y una es implícita.

Las seis marcadas como **inalcanzables** no constituyen un olvido: son aserciones sobre salidas del DUT, y el entorno de verificación no puede violarlas. La justificación completa se expone en el apartado III-E.1, y la recomendación de abordarlas mediante un DUT mutado, en el apartado VIII.

2) *Funcionalidades:* Tabla XXXII.

Las dos notas son las del plan de verificación: F-03 no dispone de punto de cobertura propio porque el orden se comprueba en cada transacción y no admite un bin que distinga escenarios, y F-10 no tiene propiedad asociada porque la vuelta de los punteros no es observable desde las interfaces.

3) *Comprobación de huérfanos:* La matriz sirve para acreditar, pero su función más útil es **delatar lo que falta**. Las cuatro preguntas pertinentes se responden en la tabla XXXIII:

Ninguna casilla queda en blanco sin una razón escrita, y esa es la afirmación que la matriz permite hacer de forma verificable.

VII. LIMITACIONES DEL ENTORNO Y DECISIONES TOMADAS

Documentar una limitación de la herramienta, junto con la vía alternativa que se adoptó, demuestra criterio; ocultarla, no.

A. Limitaciones de XSim

La tabla XXXIV recoge las limitaciones del simulador que condicionaron decisiones de implementación.

La primera merece detalle porque afecta al uso diario. En XSim los plusargs existen únicamente en la tabla de `$value$plusargs` y nunca en el `argv` del proceso, mientras que `uvm_cmdline_processor` lee `argv` a través de DPI. En consecuencia, todo lo que UVM consulta con `get_arg_matches()` se ignora sin aviso. Las dos excepciones se compilan dentro de la rama sin DPI de la biblioteca.

El puente implementado en el test base traduce los argumentos relevantes a sus API equivalentes, según la tabla XXXV:

B. Decisiones condicionadas por el simulador

Las siguientes decisiones de arquitectura, descritas en el apartado IV, existen por limitaciones de la herramienta y no por preferencia de diseño:

- **Propagación de reset con llamadas explícitas** (apartado IV-E.3) en lugar del recorrido sobre `get_children()` con `$cast` a `interface class` que implementan `prim_async_fifo_reset_handler` (la clase abstracta). Ya que XSIM no la implementa.

Propiedad	Mecanismo	Evidencia en el código	Val. por inyección
PROP-01 Orden	Scoreboard	<code>uvm_error("PROP01")</code>	Sí
PROP-02 Sin pérdidas	Scoreboard	<code>uvm_error("PROP02")</code>	No (<i>TEST-08e</i>)
PROP-03 Sin duplicados	Scoreboard	<code>uvm_error("PROP03")</code>	Sí
PROP-04 Estabilidad de <i>valid</i>	SVA $\times 2$	PROP04_WVALID_STABLE_A y análoga en lectura	No (<i>TEST-08h</i>)
PROP-05 Estabilidad del dato	SVA $\times 2$	PROP05_WDATA_STABLE_A y análoga en lectura	No (<i>TEST-08i</i>)
PROP-06a Sin petición en reset	SVA	PROP06A_NO_REQUEST_DURING_RESET_A	Sí
PROP-06b Estado observable (escribible)	SVA	PROP06B_DEPTH_ZERO_DURING_RESET_A	Inalcanzable
PROP-06c Sin dato ofrecido en reset	SVA	PROP06C_NO_VALID_DURING_RESET_A	Inalcanzable
PROP-06d Estado observable (lectura)	SVA	PROP06D_DEPTH_ZERO_DURING_RESET_A	Inalcanzable
PROP-07 Conservadurismo de la ocupación	Scoreboard	<code>uvm_error("PROP07")</code>	Sí, ambas direcciones
PROP-08 Cota superior de los reportes	SVA $\times 2$	PROP08_WDEPTH_LEQ_DEPTH_A y análoga en lectura	Inalcanzable
PROP-09 Tolerancia a la latencia	Implícita	El modelo no asume latencia	No ejercitada
PROP-10 Bloqueo por lleno	SVA	PROP10_NO_READY_WHEN_FULL_A	Inalcanzable
PROP-11 Bloqueo por vacío	SVA	PROP11_NO_VALID_WHEN_EMPTY_A	Inalcanzable
PROP-12 Reset	Scoreboard y vigilante del entorno	<code>handle_reset()</code> , <code>uvm_error("RESET_ERROR")</code>	Sí
PROP-13 Liberación síncrona	Inmediata $\times 2$	<code>\$error("PROP-13: ...")</code> en ambas interfaces	Sí

TABLE XXXI

TRAZABILIDAD DE LAS PROPIEDADES: MECANISMO DE COMPROBACIÓN, EVIDENCIA LOCALIZABLE EN EL CÓDIGO Y ESTADO DE VALIDACIÓN POR INYECCIÓN.

- **Renombrado de la señal interna del driver** a `wvalid_drv` (apartado IV-C.4) en lugar de un alias en el elemento del clocking block, que es SystemVerilog válido pero no está implementado.
- **Puente de argumentos de línea de comandos** en el test base (apartado IV-G.4) en lugar de los plusargs nativos de UVM.
- **Volcado propio de jerarquía** con `get_children()` en lugar de `print_topology()`.

1) *Un único simulador*: El trabajo se desarrolló inicialmente sobre dos flujos paralelos, uno para XSim y otro para un simulador comercial (DSIM), con el mismo banco de pruebas y la misma lista de fuentes. La razón no era la portabilidad sino el diagnóstico: cuando un comportamiento resulta sorprendente, ejecutarlo en un segundo simulador **distingue un defecto propio de una limitación de la herramienta**, que es la duda que aparece con más frecuencia al trabajar contra un simulador con restricciones conocidas.

El flujo doble se probó y funcionó, y varias de las limitaciones de la tabla XXXIV se identificaron precisamente así. Se abandonó por falta de licencia de DSIM, no por falta de utilidad, y el trabajo se cerró sobre un único flujo (XSIM). Se deja constancia porque la técnica es recomendable siempre que haya licencias disponibles, y porque explica el origen de parte de la tabla XXXV.

Un caso que quedó sin resolver por esta causa. Declarar una señal como `inout` en un clocking block, para conducirla y muestrearla con el mismo objeto, produjo X

en el muestreo (apartado IV-C.5). El comportamiento se caracterizó lo bastante como para evitarlo y para entender el síntoma que causaba, pero **no se determinó si se ajusta a lo que exige el estándar** [3] o si es una particularidad de la herramienta. Es exactamente la duda que una segunda ejecución habría despejado en minutos, y no figura en la tabla XXXIV precisamente porque no se pudo acreditar como limitación. La solución adoptada, separar la petición del pin, es en todo caso preferible con independencia de cuál sea la respuesta.

C. El simulador no señala el fallo de un test

`xsim` devuelve **código de salida cero en todos los casos**. Se verificó experimentalmente sobre tres escenarios: corrida correcta, corrida terminada con `UVM_FATAL` y petición de un test inexistente. Los tres devuelven cero.

En consecuencia, un script de regresión construido sobre el código de salida, como el que se empleó inicialmente en este proyecto, **no puede detectar ningún fallo**. Una construcción de la forma `make run || exit 1` nunca llega a activar su rama de error.

El análisis del fichero de registro es la vía estándar en la industria y no un rodeo: los simuladores no ofrecen un canal fiable para comunicar que un test ha fallado. Lo que distingue un criterio sólido de uno ingenuo es que **la ausencia de errores no se toma como éxito**. Un registro truncado, un simulador que ha terminado de forma anómala o un test que no llegó a arrancar producen todos un registro sin errores.

Id	Función	Propiedad	Cobertura	Final	Tests que la ejercitan
F-01	Escritura con handshake	PROP-04, 05, 10	CP-01, CP-04	100 %	TEST-01, 02, 04
F-02	Lectura con handshake	PROP-04, 05, 11	CP-02	100 %	TEST-01, 03, 04
F-03	Preservación del orden	PROP-01, 02, 03	Nota 1	—	Todos
F-04	Condición de lleno	PROP-10	CP-06, CP-14	100 %	TEST-02
F-05	Condición de vacío	PROP-11	CP-07, CP-15	100 %	TEST-03
F-06	Ocupación en escritura	PROP-07, 08	CP-04, CP-12	100 %	TEST-01, 02
F-07	Ocupación en lectura	PROP-07, 08	CP-05, 07, 12	100 %	TEST-03
F-08	Relojes de frecuencia arbitraria	PROP-09	CP-10 a CP-15	100 %	TEST-05 y los dirigidos bajo cada clase
F-09	Reset simultáneo de ambos dominios	PROP-06, 12, 13	CP-03, CP-09	100 %	TEST-06
F-10	Vuelta de los punteros	Nota 2	CP-08	100 %	TEST-01, 04

TABLE XXXII

TRAZABILIDAD DE LAS FUNCIONALIDADES: PROPIEDADES ASOCIADAS, COBERTURA Y CIERRE ALCANZADO.

Pregunta	Respuesta
¿Alguna propiedad sin mecanismo de comprobación?	No, pero PROP-09 queda sin ejercitar : el retardo aleatorio del cruce no se compila y la latencia es constante (apartado III-D.1)
¿Algún punto de cobertura sin test asignado?	No. La trazabilidad inversa los asigna todos
¿Algún test que no cierre nada?	TEST-08, declarado opcional: su propósito es validar comprobadores, no cubrir puntos
¿Alguna funcionalidad sin propiedad ni cobertura?	No. F-03 y F-10 carecen de una de las dos, con justificación explícita

TABLE XXXIII

COMPROBACIÓN DE HUÉRFANOS SOBRE LA MATRIZ DE TRAZABILIDAD.

1) *Marcador positivo emitido por el banco*: El criterio adoptado invierte la lógica: el banco emite al terminar un **marcador positivo**, y su **ausencia** se interpreta como fallo, sea cual sea la causa. El script de regresión no busca errores, busca el marcador.

La decisión se toma en la última fase del test base, donde se reúnen las tres fuentes de fallo posibles:

```

virtual function void
final_phase(uvm_phase phase);
...
n_errors = svr.get_severity_count(
    UVM_ERROR);
n_fatals = svr.get_severity_count(
    UVM_FATAL);
// las aserciones no pasan por el
// report server
n_assert_fails =
    env.wr_agent.vif.assert_fail_count +
    env.rd_agent.vif.assert_fail_count;

if (run_passed(n_errors, n_fatals,
    n_assert_fails))
    $display("***_TEST_PASSED_***");
else
    $display("***_TEST_FAILED_***");
endfunction

```

El criterio se aísla en un método virtual porque la familia de inyección de fallos del apartado III-E.1 lo **invierte**: allí el test pasa cuando el comprobador dispara, y una corrida limpia constituye el fallo.

2) *Validación del propio marcador*: Un marcador que solo se ha visto decir "aprobado" no está verificado, de modo que se comprobó reintroduciendo temporalmente un defecto conocido: la variante de PROP-13 que almacenaba el instante del flanco en un tipo integral (defecto 7 del apartado VI-B). El resultado es la demostración exacta del problema que el marcador resuelve:

```

uvm_errors=0 uvm_fatals=0
assertion_failures=1
*** TEST FAILED ***

UVM_ERROR : 0

```

El resumen de UVM informa de cero errores y el veredicto declara la corrida fallida. Con el criterio anterior, basado en leer ese resumen, el defecto habría sobrevivido a la regresión, como en efecto sobrevivió a varias fases del desarrollo.

3) *Las aserciones no pasan por el sistema de reporte*: Las aserciones SVA emiten \$error y **no incrementan el contador UVM_ERROR**. Se decidió no encaminarlas al sistema de reporte de UVM porque ello obligaría a la interfaz a importar el paquete de UVM, y las interfaces se emplean también en bancos sin UVM (así se validaron las trece propiedades antes de que existiera el entorno), además de ser un requisito para un eventual flujo formal.

Esa decisión deja un hueco: una corrida con aserciones disparando termina informando de cero errores. Se cierra con un **contador entero en cada interfaz**, incrementado en el bloque de acción de cada una de las catorce comprobaciones

Limitación	Consecuencia	Solución adoptada
Los plusargs no llegan a <code>uvm_cmdline_processor</code>	Se ignoran silenciosamente todos salvo <code>UVM_TESTNAME</code> y <code>UVM_VERBOSITY</code>	Puente con <code>\$value\$plusargs</code> en el test base
<code>print_topology()</code> imprime solo la raíz	No hay volcado de jerarquía	Recorrido propio con <code>get_children()</code>
<code>\$cast</code> a <i>interface class</i> no implementado	No es posible recorrer los hijos buscando un contrato	Llamadas explícitas en el agente
Alias en elementos de clocking block (<code>output a = b;</code>)	No compila pese a ser SystemVerilog válido [3]	Renombrar la señal en el driver
Backdoor de RAL por <code>hdl_path</code> mediante VPI	No disponible	No aplica: el DUT no expone registros

TABLE XXXIV
LIMITACIONES DE XSim QUE CONDICIONARON LA IMPLEMENTACIÓN.

Plusarg	Equivalente invocado
<code>UVM_TIMEOUT</code>	<code>uvm_root::get().set_timeout()</code>
<code>UVM_MAX_QUIT_COUNT</code>	<code>set_max_quit_count()</code> sobre el <i>report server</i>
<code>UVM_CONFIG_DB_TRACE</code>	<code>uvm_config_db_options::turn_on_tracing()</code>
<code>UVM_RESOURCE_DB_TRACE</code>	<code>uvm_resource_db_options::turn_on_tracing()</code>

TABLE XXXV
PUENTE DE ARGUMENTOS DE LÍNEA DE COMANDOS IMPLEMENTADO EN EL TEST BASE.

junto al `$error`. Un entero no obliga a la interfaz a importar nada, de modo que la decisión original permanece intacta, y el test base lo lee por la interfaz virtual para emitir su veredicto.

El mismo contador cumple una segunda función: es el mecanismo por el que la familia de inyección de fallos observaría el disparo de las aserciones, que un interceptor de informes de UVM no puede ver.

4) *Una tercera fuente que el marcador no ve:* El contador anterior cubre las aserciones **de las interfaces propias**. Queda fuera una tercera fuente de error: **las aserciones que el propio diseño lleva dentro**. `prim_fifo_async` incluye las suyas, y emiten por el simulador sin pasar por ningún contador del entorno ni por el sistema de informes de UVM.

El hueco no es teórico. Ante un diseño con el codificador Gray sustituido por un contador binario, esas aserciones disparan **128 veces** en una sola corrida de TEST-04 y el banco emite `*** TEST PASSED ***`. El apartado VI-E recoge la medida.

Dónde se recoge, y por qué no en el marcador. Incorporarlas exigiría que el banco pudiera contarlas, y no hay vía inmediata: no son suyas, no puede instrumentar su bloque de acción como hace con las propias, y encaminarlas por UVM obligaría al RTL del diseño a conocer el entorno de verificación. El guion de regresión las localiza en el registro, que es la vía que el apartado VII-C ya defiende como estándar.

Pero no deciden el aprobado, y la razón es de alcance antes que de mecanismo: no se derivaron del contrato, no están entre las trece propiedades ni en la matriz de trazabilidad, y el criterio de cierre del apartado III-F no las declara. Hacerlas vinculantes sería ampliar el criterio por la puerta de atrás. Se cuentan y se avisan, con la cifra a la vista: un disparo suelto tiene explicación benigna, decenas no.

La lección que deja es de inventario antes que de mecan-

ismo: **un comprobador cuyo disparo no llega al veredicto equivale a no tenerlo**, y para saber cuáles llegan hay que empezar por saber cuáles existen.

D. Agregación de cobertura sin herramienta de merge

`xrcg`, la herramienta de Vivado que consolida bases de datos de cobertura entre corridas, requiere una licencia de la que no se dispuso. Sin ella no existe forma de fusionar la cobertura de una regresión con las herramientas del simulador.

A esa carencia se suma una segunda, esta del lenguaje y no del simulador: **SystemVerilog no ofrece ninguna API para enumerar los bins de un covergroup ni sus aciertos** [3]. El método `get_inst_coverage()` devuelve un porcentaje, y un 75 % indica que falta uno de cuatro bins sin decir cuál. El criterio de aprobado necesita lo segundo, porque un test dirigido debe acreditar que llenó *el* bin del que es responsable y no que alcanzó cierto porcentaje.

La vía adoptada resuelve ambas a la vez. Cada componente de cobertura lleva una contabilidad por bin en paralelo al *covergroup*, la vuelca a un fichero CSV en *extract_phase*, y un script en Python consolida los ficheros de todas las corridas sumando aciertos y recalculando los porcentajes. La contabilidad reside en una base común a los tres componentes, descrita en el apartado IV-E.5.

1) *El autochequeo de la contabilidad:* El riesgo de llevar una contabilidad propia es la **duplicación**. Existen entonces dos clasificaciones del mismo valor, la del *covergroup* y la propia, que pueden divergir sin que nadie lo advierta. Por eso cada corrida compara en *check_phase* el porcentaje derivado de los contadores con el que reporta el *covergroup*, y falla si discrepan. La contabilidad queda así verificada en lugar de supuesta, y la comprobación detectó dos divergencias reales durante la construcción del entorno.

Los bins se predeclaran completos, de modo que uno sin aciertos aparezca en el informe con valor cero y no como ausente. La distinción es la que separa «no se alcanzó» de

«nadie lo midió», y sin ella el consolidado carecería de denominador: sería imposible saber si un punto tiene tres bins o cuatro, ni nombrar el que falta.

2) *Los porcentajes de cobertura no se promedian*: Es el error clásico al consolidar una regresión y conviene dejarlo advertido. Los porcentajes **no se promedian** entre corridas, se recalculan sobre los aciertos consolidados. Dos corridas al 50 % no dan un 50 % conjunto si cada una alcanzó un bin distinto. Es especialmente relevante para los cruces CP-12 a CP-15, cuyo techo por corrida es de un tercio: promediarlos daría permanentemente un tercio y ocultaría que la regresión sí los cierra.

E. Alcance no cubierto

El plan de verificación declaró cuatro exclusiones antes de empezar. Al cerrar el trabajo las cuatro se mantienen, y a ellas se suma una quinta que apareció por el camino, como recoge la tabla XXXVI.

Declarado fuera de alcance	Estado al cierre
Resolución de metaestabilidad	Se mantiene: es inverificable por simulación RTL , con independencia de la calidad del banco
Verificación estructural de CDC	Se mantiene: requiere herramienta de análisis estático
Análisis temporal, síntesis e implementación física	Se mantiene: el trabajo se limita al nivel RTL
Verificación formal	Se mantiene por alcance temporal
Automatización de la inyección de fallos	Nueva : la técnica se aplicó de forma manual y sus resultados constan en el apartado VI-C; su conversión en tests permanentes no se abordó por tiempo

TABLE XXXVI

EXCLUSIONES DECLARADAS Y SU ESTADO AL CIERRE DEL TRABAJO.

El modelo de registros no llegó a ser una exclusión sino una **no aplicabilidad**: el diseño no expone registros ni bus de configuración, de modo que no hay nada que modelar.

Conviene distinguir qué exclusiones son de **naturaleza** y cuáles de **alcance**. Las dos primeras no se resolverían con más tiempo, pues exigen otras clases de herramienta. Las tres últimas sí, y por ese motivo las tres reaparecen como recomendaciones en el apartado VIII en lugar de quedarse únicamente aquí.

VIII. CONCLUSIONES Y TRABAJO FUTURO

A. Objetivos alcanzados

Los cinco objetivos específicos enunciados en el apartado I-B, contrastados con lo obtenido en la tabla XXXVII:

Los cinco puntos del criterio de cierre del apartado III-F se cumplen igualmente:

- 1) **100 % sobre los puntos CP-01 a CP-10**: alcanzado.
- 2) **Al menos un 90 % sobre los cruces CP-12 a CP-15**: alcanzado el **100 %**, sin ningún bin justificado por inalcanzabilidad.
- 3) **Cero fallos en la regresión**: 54 corridas, cero fallidas.
- 4) **Las trece propiedades activas durante la regresión**: acreditado en la matriz de trazabilidad, con la etiqueta

Objetivo	Estado	Evidencia
1. Derivar una especificación verificable del DUT	Cumplido	Trece propiedades y diez funcionalidades derivadas del contrato de interfaz y de la referencia clásica [1], no del RTL
2. Testbench UVM con dos agentes independientes	Cumplido	Apartado IV
3. Comprobación automática mediante scoreboard y SVA	Cumplido	Catorce comprobaciones de protocolo y cinco en el scoreboard
4. Plan de cobertura con criterio de cierre alcanzado	Cumplido	66 de 66 bins, 100 % (apartado VI-G)
5. Documentar proceso, decisiones y limitaciones	Cumplido	Apartados IV a VII

TABLE XXXVII

OBJETIVOS ESPECÍFICOS Y EVIDENCIA DE SU CUMPLIMIENTO.

de cada aserción como evidencia localizable en el código.

5) **Todos los tests obligatorios en verde**: los seis del catálogo.

El criterio se cumple en sus cinco puntos, y los umbrales son los que se fijaron *antes* de construir el banco: ninguno se rebajó después. La afirmación conviene hacerla comprobable, porque el plan sí cambió durante la implementación, en cinco puntos que se enumeran con su dirección:

- **CP-03 se desdobló en CP-03a y CP-03b**, uno por agente. Es una cuestión de granularidad en el registro: se mide lo mismo, anotado por dominio.
- **Los bins empty de CP-04 y full de CP-05 se eliminaron**. Se retiraron por inalcanzabilidad estructural demostrada, no por no haberse alcanzado, y el razonamiento consta en el apartado III-D.1. Un bin que no puede llenarse mantiene la cobertura por debajo del 100 % de forma permanente y convierte el criterio en inservible.
- **El centinela de reset en los bins de transición** (apartado IV) **endurece** la medición: rechaza como acierto una transición que el covergroup daba por buena, porque abandonar la condición de lleno *por efecto del reset* no demuestra lo que el bin existe para demostrar.
- **El significado de en verde pasó de cero errores de UVM al marcador positivo** (apartado VII-C), lo que también **endurece**: al menos una corrida que aprobaba con el criterio anterior falla con el nuevo, y ninguna al revés.
- **PROP-06 se subdividió en cuatro subpropiedades** al escribirla como aserción, por las razones del apartado V-B.3. El contrato no se debilitó; se hizo explícito quién es el sujeto de cada obligación.

Dos de los cinco hacen la comprobación más estricta, dos son de forma y el quinto retira lo que se demostró imposible. **Lo que no ocurrió en ningún caso fue ajustar la definición de un bin para que se llenase**, que es la forma habitual de alcanzar un criterio sin cumplirlo, y que el propio apartado III-F declara inadmisibile.

1) *Lo que el criterio no afirma:* Que la cobertura esté al 100% significa que se visitaron todas las situaciones **que el plan declaró interesantes**. Es por tanto una afirmación sobre el plan tanto como sobre el diseño, y de ahí que el apartado III dedique más espacio a justificar cada punto de cobertura que a enumerarlos.

B. Lecciones metodológicas

- **Escribir las aserciones de protocolo antes que los drivers.** De los siete defectos del entorno, **seis no alteraban ningún dato**: una comparación de valores los habría dado por correctos, porque lo que fallaba era la forma de onda del protocolo. Las aserciones ya existían cuando se escribió el driver, y cazaron el defecto 2 antes de que hubiera nada con qué comparar.
- **Validar los comprobadores, no solo el diseño.** Cada aserción y cada rama del scoreboard se enfrentó a un fallo inyectado que demuestra que detecta lo que dice detectar. La práctica nació de un incidente: doce aserciones estuvieron desactivadas en silencio por un bit sin inicializar, y la simulación pasaba en verde.
- **Un contrato ambiguo produce comprobadores ambiguos.** PROP-06 y PROP-12 hubo que reescribirlas al implementarlas, no por un error de redacción sino porque el enunciado original no identificaba **el sujeto** de la obligación, agente o diseño, ni qué casos quedaban fuera del contrato.
- **El cierre de cobertura determinista no equivale al probabilístico.** Un test aleatorio con dos mil transacciones por corrida dejó sin cerrar un punto en tres de cada cinco semillas. Un bin alcanzado por azar puede faltar en la siguiente corrida sin que nada haya cambiado, de modo que solo el cierre que un test dirigido garantiza en toda corrida es reclamable como tal.
- **Una declaración sin consumidor es código no verificado.** El simulador no analiza el cuerpo de un método hasta que algo instancia la clase, ni el de una `property` hasta que una aserción la invoca.
- **Las infraestructuras de comprobación deben fallar en cerrado.** El valor por defecto de un interruptor de comprobaciones es *activo*; el de una espera ante un valor desconocido es *esperar*.
- **Distinguir restricción de comprobación.** Restringir el estímulo y afirmar la misma condición no son medidas redundantes: la primera describe el entorno propio, la segunda detecta cuándo alguien, el testbench o el bloque vecino en integración, no lo respeta.
- **La disciplina del clocking block no admite excepciones parciales.** La regla está en el estándar [3]; lo que se midió, y solo en XSim, es su modo de fallo: una única asignación directa a una señal conducida por un clocking block lo inutiliza para el resto de la simulación.
- **Una propiedad «implícita» necesita cobertura más que las demás.** PROP-09 se declaró verificada de forma implícita, por no modelar el scoreboard la latencia del cruce. Las otras doce tienen una aserción o una

rama del comprobador cuyo silencio prolongado resulta sospechoso; esta no tenía nada, y por eso pudo pasar cuatro semanas sin ejercitarse sin que nada lo delatara. Un punto de cobertura sobre el desfase observado habría mostrado un bin vacío desde la primera regresión.

- **Distinguir la regla conocida del hallazgo propio.** Que una señal no admita a la vez conducción por clocking block y asignación procedural está en el estándar [3]; aquí se llegó a ello construyendo un banco aislado para medirlo. Lo único que la medición añadía era el modo de fallo, silencioso y permanente, que el estándar no fija. Reconocer antes cuándo una dificultad es un problema ya resuelto, y reservar la medición para lo que no lo está, separa avanzar de redescubrir.

C. Trabajo futuro y recomendaciones

1) *Validación sistemática de los comprobadores:* **Es la recomendación de mayor prioridad para un trabajo que continúe este**, y no se cubrió aquí por alcance temporal.

Un comprobador que nunca ha fallado no está verificado. En este proyecto la distinción resultó no ser teórica en dos ocasiones distintas:

- Las doce aserciones de protocolo estuvieron **desactivadas en silencio** durante una fase entera, por un interruptor sin inicializar. La simulación pasaba en verde porque no se comprobaba nada (defecto 4 del apartado VI-B).
- PROP-13 **acusó al diseño de un incumplimiento inexistente** durante varias fases, mientras el resumen de UVM informaba de cero errores (defecto 7 del mismo apartado).

Ninguno de los dos lo habría detectado una regresión en verde. Los dos aparecieron al presentar deliberadamente al banco la violación que decía detectar.

Lo que este trabajo sí hizo. Seis inyecciones manuales, una por comprobador, con sus resultados tabulados en el apartado VI-C. Cada una introdujo un defecto conocido en el entorno y verificó que el comprobador correspondiente disparaba, y que contra el diseño real permanecía en silencio. Eso demuestra que el conjunto **discrimina** entre comportamiento correcto e incorrecto.

Lo que queda pendiente, y por qué importa. Las inyecciones de errores se realizaron sobre copias temporales del árbol de ficheros y no sobrevivieron para ser parte de la regresión. Una inyección que existió una vez demuestra que el comprobador funcionaba *ese día*, pero no protege frente a que un cambio posterior lo desconecte. Convertirlas en tests permanentes dentro de la regresión es lo que transforma una comprobación puntual en una garantía sostenida, y es exactamente el tipo de trabajo cuyo valor solo se aprecia meses después.

El diseño está especificado en el apartado III-E.1 como la familia TEST-08, con el catálogo de defectos y los dos mecanismos que requiere. No se implementó por falta de tiempo, y se deja constancia de ello en lugar de omitir el punto: **reconocer una carencia con su justificación es información para quien continúe; suprimirla no lo es.**

Un límite encontrado al especificarla. De las catorce comprobaciones de protocolo, **seis no son validables por esta técnica**: son aserciones sobre salidas del DUT, y el entorno de verificación no puede violarlas, pues ni el driver ni el monitor pueden hacer que `wready_o` suba con el FIFO lleno. Exigirían un DUT mutado, que constituye una técnica distinta con su propio coste de mantenimiento. Un trabajo que aborde esta recomendación debería decidir primero si asume esa segunda técnica o si acota el alcance a los comprobadores del entorno.

Conviene además no perder un hallazgo colateral: **el modelado que garantiza una propiedad puede impedir inyectar su violación**. El gateado asíncrono de `wvalid_i` en la interfaz, introducido para garantizar PROP-06a, hace estructuralmente imposible que el agente afirme esa señal durante el reset. La inyección que validaba la propiedad dejó de ser alcanzable al mejorar el banco.

2) *Otras líneas*: Por orden de lo que más añadiría a este trabajo:

- **Verificación formal de PROP-01 a PROP-03.** Orden, ausencia de pérdidas y ausencia de duplicados son propiedades de seguridad sobre un espacio de estados acotado, el caso de libro para una demostración formal. La simulación las comprueba sobre las trazas que se ejecutan; una prueba formal las establecería para todas.
- **Análisis estructural de CDC.** Comprueba que todo cruce dispone de sincronizador y que no existen rutas de convergencia peligrosas, sin depender de que una simulación reproduzca el fenómeno. Es la vía que puede pronunciarse sobre la metaestabilidad, declarada inverificable por simulación en el apartado III-G. No sustituye a lo que el banco ya hace: el apartado VI-E mide que un puntero sin codificación Gray se detecta en simulación, y de inmediato. Lo que añade es cobertura *exhaustiva* de los cruces, que una campaña dirigida no garantiza.
- **Parametrización efectiva.** El banco está escrito para tomar la profundidad y la anchura como parámetros, pero toda la verificación se ejecutó con una única geometría. Barrer varias validaría el propio banco además del diseño, y algunos bins, como el de ocupación a la mitad, dejarían de asumir una profundidad mínima.
- **Cobertura de aserciones.** Saber con qué frecuencia se evaluó cada aserción, y no solo que no falló, distingue una propiedad vigilante de una que nunca llegó a ejercitarse. El apartado V-B.4 expone el caso concreto que esta técnica detecta y ninguna otra.
- **Virtual sequencer.** No llegó a ser necesario, pues la coordinación entre dominios se resolvió con métodos auxiliares en el test base. Se menciona porque la decisión de no aplicarlo fue deliberada y su disparador está escrito: cuando un escenario multidominio deba reutilizarse como secuencia y no como test.

3) *Técnicas identificadas y no aplicadas*: Se enumeran explícitamente para que conste que su omisión fue una decisión y no un desconocimiento:

- **Modelo de registros (RAL).** El diseño no expone registros ni bus de configuración, de modo que no hay nada que modelar.
- **Instance override.** No existe más de una instancia de ningún tipo en el entorno, de manera que un *type override* ya afecta a un solo dominio.
- **Construcción de VIP reutilizable.** Los dos agentes cumplen la estructura, pero no se abordó la disciplina de empaquetado y distribución, que solo se justifica ante un consumidor externo.

USO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

En la realización de este trabajo se empleó el modelo de lenguaje **Claude Opus 5**, de Anthropic, a través de la interfaz Claude Code. Se declara aquí su uso por transparencia y porque la naturaleza de ese uso condiciona cómo debe interpretarse la autoría del trabajo.

Para qué se empleó. Consulta de prácticas habituales en la aplicación de UVM; resolución de dudas sobre el estándar SystemVerilog y sobre alternativas de implementación; diagnóstico de comportamientos del simulador; revisión de forma y estructura de este documento; y apoyo en la organización del trabajo a lo largo del proyecto.

Qué condición tiene. La herramienta se trabajó como tal: un instrumento de consulta y de redacción, no una fuente de autoridad. No se le reconoce autoría, no se la cita como referencia bibliográfica y sus respuestas no se consideraron válidas por sí mismas. Un modelo de lenguaje produce texto plausible, no texto verificado, y en un trabajo de verificación esa distinción es precisamente la que está en juego.

Cómo se controló. Rige el mismo criterio que el resto del trabajo: **toda afirmación se contrasta antes de darla por buena**. Las decisiones de metodología, la especificación de propiedades, el plan de cobertura y el criterio de cierre son del autor. Las afirmaciones sobre el lenguaje se contrastaron contra el estándar [3]; las referidas al comportamiento del simulador, ejecutándolo; y las relativas al diseño, sobre el RTL. Varias indicaciones de la herramienta resultaron erróneas y se corrigieron por esa vía, lo que ilustra el motivo de la cautela mejor que cualquier declaración de principios.

De quién es la responsabilidad. Del autor, íntegramente, incluidos los errores que puedan subsistir.

REFERENCES

- [1] C. E. Cummings, "Simulation and Synthesis Techniques for Asynchronous FIFO Design," in *Proc. Synopsys Users Group (SNUG)*, San Jose, CA, 2002.
- [2] C. E. Cummings and D. Mills, "Synchronous Resets? Asynchronous Resets? I am so confused! How will I ever know which to use?," in *Proc. Synopsys Users Group (SNUG)*, San Jose, CA, 2001.
- [3] *IEEE Standard for SystemVerilog; Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.
- [4] Accellera Systems Initiative, *Universal Verification Methodology (UVM) 1.2 Class Reference*, 2014.
- [5] lowRISC, *OpenTitan: Open source silicon root of trust*. [Online]. Available: <https://github.com/lowRISC/opentitan>